

Joel Martin

Lokaalin kielimallin sekä Retrieval-Augmented Generation -menetelmän hyödyntäminen chattibotissa

Metropolia Ammattikorkeakoulu

Insinööri (AMK)

Tieto- ja viestintätekniikka

Insinöörityö

4.4.2025

Tiivistelmä

Tekijä:	Joel Martin
Otsikko:	Lokaalin kielimallin sekä Retrieval-Augmented Generation -menetelmän hyödyntäminen chattibotissa
Sivumäärä:	41 sivua
Aika:	4.4.2025
Tutkinto:	Insinööri (AMK)
Tutkinto-ohjelma:	Tieto- ja viestintätekniikka
Ammatillinen pääaine:	Ohjelmistotuotanto
Ohjaajat:	Lehtori Vesa Ollikainen Tutkijaopettaja Aarne Klemetti

---

Insinöörityön tavoitteena oli tutkia lokaalien kielimallien hyödyntämistä yrityksen tarpeisiin tehdyssä sovelluksessa. Tilaajaksi määriteltiin hypoteettinen yritys, jonka motiivina käyttää lokaaleja kielimalleja oli yrityksen sensitiivisen sisäisen datan pitäminen yksityisenä. Yritys halusi tietää, voidaanko lokaaleilla kielimalleilla tehdä hyödyllinen sovellus, jota voitaisiin ajaa yrityksen käyttämillä työkannettavilla. Insinöörityötä varten tutkittiin yleisesti kielimallien hyötyjä ja rajoitteita sekä lokaalien kielimallien ja pilvimallien eroja.

Insinöörityössä luotiin yksinkertainen sovellus, joka täyttää projektille asetetut vaatimukset. Kielimallin lokaaliin ajamiseen valittiin Ollama-ohjelma. Sovellukseen rakennettiin Retrieval-Augmented Generation -logiikka hyödyntämällä LangChain-kirjastoa. Lisäksi luotiin käyttöliittymä, joka lähettää generointipyyntöjä ja palvelin välittämään dataa. Sovelluksen toteutukseen valitut teknologiat osoittautuivat sopiviksi ja helppokäyttöisiksi kehityksen aikana.

Sovellus täytti asetetut vaatimukset hyvin, mutta ongelmaksi muodostui suoritusnopeus heikommalla laitteistolla. Yrityksen työkannettavien suorituskyky arvioitiin jonnekin koeajoissa käytettyjen laitteiden suorituskykyjen välille. Ratkaisuksi esitettiin uudemman laitteiston ostaminen sekä generoinnin siirtäminen erilliselle palvelinlaitteelle. Yrityksen arvioitavaksi jäi, oliko suoritusnopeus riittävä vai tarvitaanko investointeja uuteen laitteistoon. Sovelluksen koeajojen tulokset osoittivat, että lokaaleja kielimalleja voidaan jo käyttää yritykselle hyödyllisessä sovelluksessa. Sovellus tarjoaa pohjan, jota on helppo laajentaa vaihtoehtona pilvipohjaisille ratkaisuille.

Insinöörityöraporttia kirjoittaessa käytettiin OpenAI:n ChatGPT:tä, jonka pohjana oleva kielimalli oli GPT-4-turbo (April 2024). Tekoälyä käytettiin harvoissa tapauksissa apuna vakiintumattoman sanaston kääntämisessä, löytämään uusia näkökulmia aiheisiin ja uusien lähteiden etsinnässä internetistä.

Avainsanat: Tekoäly, LLM, Kielimallit, LangChain, RAG

Tämän opinnäytetyön alkuperä on tarkastettu Turnitin Originality Check -ohjelmalla.

Abstract

Author:	Joel Martin
Title:	Utilizing Local Large Language Models and the Retrieval-Augmented Generation Technique in a Chatbot
Number of Pages:	41 pages
Date:	4 April 2025
Degree:	Bachelor of Engineering
Degree Programme:	Information and Communication Technology
Professional Major:	Software Engineering
Supervisors:	Vesa Ollikainen, Senior Lecturer Aarne Klemetti, Researching Lecturer

---

The object of the study was to research the potential of utilizing local large language models from the perspective of a company. A hypothetical client company was devised. The company's motivation for using local language models was to keep its sensitive internal data private. The company wanted to determine whether a useful application that would run on the company's work laptops could be built with local language models. As part of the study, the benefits and limitations of language models were examined, as well as the differences between local language models and cloud-based models.

A simple application was developed to meet the project requirements. A piece of software called Ollama was chosen for running the language model locally. The application implemented Retrieval-Augmented Generation using the LangChain library. Additionally, a user interface was created to send generation requests, and a server was implemented to handle data transmission. The selected technologies proved to be suitable and easy to use during the development process.

The application met the defined requirements well, but performance on lower-end hardware became an issue. The performance of the company's work laptops was estimated to fall somewhere between the performance levels of the devices used in the test runs. As a solution, purchasing newer hardware or offloading the generation process to a separate server was proposed. It was left for the company to evaluate whether the performance was sufficient or if investments in new hardware were necessary. The result of the test runs of the application showed that local language models can already be used in useful applications for companies. The application provides a foundation that can be easily expanded as an alternative to cloud-based solutions.

While writing this thesis, OpenAI's ChatGPT based on the large language model GPT-4-turbo (April 2024) was used as a helpful tool. Artificial intelligence was used to translate unestablished terminology, to find new perspectives on topics and to find new sources from the internet.

Keywords: AI, LLM, Large Language Models, LangChain, RAG

Sisällys

Lyhenteet

1 Johdanto 1

2 Teknologiat 2

2.1 Tekoäly 2
2.2 Kielimallit 5
2.2.1 Syötteet 7
2.2.2 Rajoitteet 9
2.3 Ollama 13
2.4 LangChain 14
2.5 Vektoritietokannat ja vektorihaku 15
2.6 Retrieval-Augmented Generation (RAG) 16

3 Lokaalit kielimallit 17

3.1 Hyödyt 18
3.2 Rajoitteet 20
3.3 Mallin valinta 21

4 Sovelluksen toteutus 25

4.1 Sovelluksen määrittely 25
4.2 Kielimallin ajaminen 26
4.3 Tiedonhaku ja generointi 27
4.4 Palvelin ja käyttöliittymä 30
4.5 Sovelluksen testaus 31

5 Tulokset ja jatkokehitys 33

6 Yhteenveto 36

Lähteet 37

Lyhenteet

AGI: *Artificial General Intelligence, yleinen tekoäly*. Tekoäly, joka kykenee suoriutumaan tehtävistä laajalla skaalalla vähintään ihmistasoisesti.

API: *Application Programming Interface*. Ohjelmointirajapinta, jota käyttämällä voidaan vaihtaa dataa kahden sovelluksen kesken.

GDPR: *General Data Protection Regulation.* Euroopan unionin yleinen tietosuoja-asetus.

LLM: *Large Language Model, suuri kielimalli.* Generatiivinen tekoälymalli, joka tuottaa luonnollisen kielen tekstiä.

MVP: *Minimum Viable Product.* Yksinkertaisin mahdollinen sovellus, joka täyttää vaatimukset.

RAG: *Retrieval-Augmented Generation*. Menetelmä, jolla kielimallin vastauksen generoinnissa hyödynnetään vektorihaulla saatua dataa.

RAM: *Random Access Memory*. Tietokoneiden keskusmuisti.

REST: *REpresentational State Transfer.* Yksi ohjelmointirajapinnan toteuttamistyyli.

VRAM: *Video Random Access Memory*. Näytönohjainten sisäinen muisti.

1 Johdanto

Tekoäly on suurten kielimallien kehittymisen myötä murroksessa. Vuoden 2022 marraskuussa alansa kehittyneimpiä kielimalleja luova yritys, OpenAI, julkaisi heidän uusinta kielimalliaan hyödyntävän ChatGPT-chattibotin, mikä toi tekoälyn suoraan kuluttajien ja yritysten käyttöön ja tietoisuuteen. ChatGPT:stä tuli kahdessa kuukaudessa nopeimmin sata miljoonaa aktiivista käyttäjää saavuttanut sovellus [1]. Kiinnostus tekoälyä kohtaan kasvoi räjähdysmäisesti, minkä johdosta erilaiset organisaatiot, yritykset ja yliopistojen tutkimusryhmät julkaisevat jatkuvasti omia kilpailevia mallejaan. Tästä syystä tekoälyn hyödyntämistä eri muodoissa yritysmaailmassa on syytä tutkia.

Insinöörityön tarkoituksena oli tutkia lokaalien suurten kielimallien käyttämistä. Tutkimus alkoi pilvessä ajettavien yksityisten yritysten kielimallien parissa, mutta tutkimustyön edetessä aihe tarkentui erityisesti lokaalien eli omalla tietokoneella ajettavien avoimien kielimallien käyttöön. Tutkimuksen edistämiseksi oli tarpeellista määritellä hypoteettinen yritys, joka toimi työn tilaajana. Yritys halusi tehostaa toimintaansa tekoälyllä. Yritys antoi tehtäväksi kartoittaa lokaalien kielimallien tuottamia mahdollisuuksia, sillä se ei halunnut lähettää sensitiivistä sisäistä dataansa pilvessä toimiviin kielimallisovelluksiin tietoturvariskin vuoksi.

Yritys asetti seuraavat tutkimuskysymykset:

- Mitä ovat lokaalit kielimallit ja miten niitä käytetään?
- Mihin käyttöön ne sopivat ja voidaanko niillä jo tehdä yritykselle hyödyllinen sovellus?
- Voidaanko kielimalleja ajaa tyypillisellä työkannettavalla vai tarvitsevatko ne jonkin muun ratkaisun?
- Millä tavoin sovellusta voitaisiin kehittää eteenpäin?

Insinöörityön tavoitteena oli siis kehittää toimiva, lokaaleja kielimalleja hyödyntävä ja yrityksen vaatimukset täyttävä prototyyppisovellus. Lisäksi työssä arvioitiin sovelluksen suorituskykyä käyttöympäristössään sekä pohdittiin jatkokehitysmahdollisuuksia.

2 Teknologiat

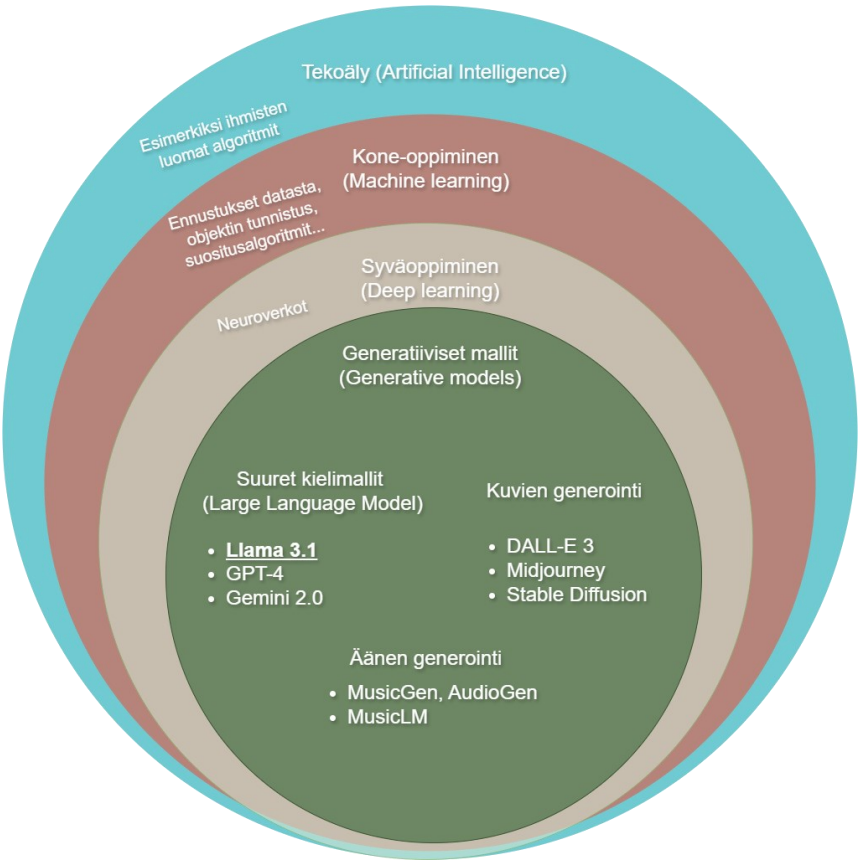
Tässä luvussa esitellään sovelluksen toteutuksessa tarvittavat käsitteet, kehitysvaiheessa testatut teknologiat sekä itse loppusovellukseen päätyneet teknologiat. Ohjelmointikieleksi projektiin valikoitui Python, sillä se on ollut perinteisesti käytössä koneoppimissovellusten kehityksessä. Python on helppokäyttöinen ja sille löytyy paljon sovelluksen luonnissa tarvitsemiani kirjastoja, mikä helpottaa kehitysprosessia huomattavasti.

2.1 Tekoäly

Tekoäly määritellään teknologiaksi, joka mahdollistaa tietokoneiden simuloivan ihmisten toimintaa. Tekoäly voi siis ihmisen tavoin oppia annetusta materiaalista, ymmärtää sitä, ratkaista ongelmia, tehdä itsenäisiä päätöksiä sekä osoittaa luovuutta. Edistyneet tekoälyt pyrkivätkin lähenemään rakenteessaan ihmisten aivoja imitoimalla neuroverkkojen rakennetta. [2.]

Korkealla tasolla tekoäly voidaan jaotella monimutkaisuuden perusteella kahteen luokkaan: vahvaan ja heikkoon. Heikko tekoäly määritellään tekoälyksi, joka on suunniteltu suorittamaan tiettyä tehtävää tai tehtäväjoukkoa. Klassisena pidetty esimerkki heikosta tekoälysovelluksesta on auto, joka kulkee määränpäähänsä turvallisesti ilman ihmiskuljettajaa. Tässä projektissa luotava sovellus kuuluu edellä mainittuun luokkaan, sillä sen tarkoitus on tuottaa tietynlaisia tekstivastauksia. Vahva tai yleinen tekoäly (engl. artificial general intelligence, AGI) tarkoittaa tekoälyä, joka suoriutuu laajalla skaalalla tehtävistä vähintään ihmisen tasolla. Tällainen tekoäly on vielä hypoteettinen, sillä sellaista ei ole saatu aikaan. [2.]

Kuvassa 1 havainnollistetaan hieman tarkemmin joitain tekoälyn tyyppejä, jotka liittyvät kielimalleihin. Kuva näyttää, mihin yläluokkiin kielimallien kaltaiset sovellukset kuuluvat. Tekoälyllä viitataan tässä luvussa moneen eri tyyppiin, mutta loput raportista keskittyy nimenomaan kielimalleihin.



Kuva 1. Tekoälyn joitain eri tyyppejä ja esimerkkisovelluksia [3; 4].

Viimeaikainen kehitys on tuonut mukanaan tekoälylle aiempaa paremmat päättelytaidot, mikä merkitsee sille huomattavasti suorituskykyisempää ongelmanratkaisua ja pidemmän ajan suunnittelukykyä. Tämän johdosta Google on esitellyt kehittämänsä tutkijoille suunnatun kumppanitekoälyn, joka on suunniteltu peilaamaan tieteellisen menetelmän päättelyprosessia. Sen on tarkoitus nopeuttaa tutkimusta, havaita puutteita tiedossa ja luoda uusia hypoteeseja sekä suuntia jatkotutkimuksille. Eräässä tapauksessa tekoälyn ehdotukset suoriutuivat asiantuntijoiden tuottamia ratkaisuja paremmin, toisessa tapauksessa tekoäly loi kahdessa päivässä saman hypoteesin, jonka luominen vei tutkijoilta kymmenen vuotta. [5.] Tekoäly ei siis näissä tapauksissa korvannut tutkijoita, vaan mahdollisti paremmat tulokset kiihdyttämällä tieteellisten löytöjen tekemistä.

Tekoälyn käyttämisen hyödyt yrityksille ovat hyvin moninaisia. Ohjelmistorobotiikalla voidaan tekoälyn ajamana automatisoida rutiininomaisia työtehtäviä ja virtaviivaistaa toimintoja, mikä parantaa tehokkuutta työpaikalla.

Tekoälyjärjestelmät voivat toimia nopeammin, tarkemmin ja säännöllisemmin kuin ihminen tietyissä tehtävissä. Samoin ne lievittävät riskiä ongelmista, jotka syntyvät ihmisten väsymyksestä ja huolimattomuudesta. Ihmiset eivät voi tehdä töitä jatkuvasti ympäri kellon, mutta tekoäly kykenee tähän ja sen vuoksi tekoälyä on valjastettu esimerkiksi chatbotteihin korvaamaan alimman tason asiakaspalvelun. Vaarallisissa ympäristöissä voidaan välttyä henkilövahingoilta käyttämällä tekoälyrobotteja. Tekoäly voi käsitellä hetkessä valtavan määrän dataa ja tuottaa siitä uutta käyttäjää kiinnostavaa tietoa. [6, s. 12–24.]

Tekoälyllä kyetään korvaamaan ihmisiä ja näin poistamaan heidän työssä tuottamiaan virheitä, mutta tämä voidaan nähdä myös huonona asia. Mitä paremmaksi tekoäly kehittyy, sitä enemmän se voi korvata ihmisiä ja näin viedä heiltä työpaikkoja. Toisaalta ihmisiä vapautuu muihin tehtäviin, mihin tekoäly ei vielä kykene. Esimerkiksi luovien alojen korvaus on hankalaa, sillä ihmisten tuottama taide on arvostetumpaa kuin tekoälyn. Tekoälyn käytön helppous ja hinta voivat kuitenkin vaikuttaa markkinoihin romahduttaen ihmisten luomien yksinkertaisimpien kuvien ja tekstien kysynnän [7]. Yritysten on siis hyvä pitää mielessä tekoälyn käyttämisen eettiset ongelmat. Liiallinen tekoälyn mahdollistama työn tehostus voi johtaa valtavaan työttömyyden nousuun. International Monetary Fundin julkaisema analyysi osoittaa, että tekoäly voi vaikuttaa edistyneimmissä maissa noin 60 prosenttiin työpaikoista sekä kasvattaa työttömyyttä ja eriarvoisuutta [8]. Useat asiantuntijat kuitenkin ovat sitä mieltä, että tulevaisuudessa tekoäly tulee olemaan enemmänkin nykyistä parempi työkalu kuin pelkästään suora ihmisten korvaaja [9; 10].

Tekoälymallien kouluttaminen ja käyttäminen voi johtaa suureen hiilijalanjälkeen, sillä etenkin suuret mallit vaativat valtavasti laskennallista tehoa. Ekologisten vaikutusten vuoksi on syytä kehittää energiatehokkaampia tekoälyalgoritmeja sekä ottaa käyttöön ekologisesti kestävämmät käytännöt tekoälyinfrastruktuurissa. [6, s. 25.] Toisaalta tekoälyn käyttö voi nopeuttaa työntekoa ja parannella prosesseja sen verran, että resursseja oikeasti säästyykin. Jevonsin paradoksin mukaan työkalujen energiatehokkuuden parantuminen kuitenkin johtaa

vääjäämättä resurssien käytön kasvuun joka tapauksessa, kun ne mahdollistavat suuremman tuotantomäärän halvemmalla [11].

2.2 Kielimallit

Lähivuosina tekoälyn kehitys on kohdistunut suurenevissa määrin generatiiviseen tekoälyyn, joka pystyy tekstimuotoisen kehotteen tai syötteen (engl. prompt) saatuaan tuottamaan vastauksena tekstiä, kuvia, koodia tai muuta audiovisuaalista sisältöä. Tällaiset tekoälymallit on koulutettu valtavalla määrällä dataa, joka yleisesti vastaa sellaista dataa, mitä mallin on tarkoitus lopuksi tuottaa. Mallit oppivat muodostamaan vastauksia, jotka ovat tilastollisesti todennäköisesti relevantteja annetun syötteen kannalta. Generatiivinen tekoäly voi siis reagoida pyyntöihin kuten taiteilijat ja kirjailijat, mutta nopeammin. Tuotetun sisällön uutuudesta ja alkuperäisyydestä voidaan väitellä, mutta joissain tapauksissa laadullisesti sisältö vastaa tai ylittää ihmisten luovat kyvyt. [12.]

Suuret kielimallit (engl. Large Language Model, LLM) ovat generatiivisen tekoälyn malleja, joiden tehtävänä on yleensä tuottaa nimenomaan luonnollisen kielen tekstiä. Luonnollinen kieli tarkoittaa ihmisten käyttämää kieltä ja puhetta. Kielimallit käyttävät eräänlaista neuroverkkoa, jota kutsutaan Transformer-malliksi. Nämä mallit ja niiden käyttämät itsehuomiointimekanismit (engl. Self-Attention) ovat syy sille, miksi kielimalleja pidetään alalla läpimurtona. Transformer-mallit voivat oppia kontekstista, mikä on tärkeää, sillä luonnolliset kielet vaativat kontekstin ymmärtämistä. Itsehuomiointimekanismia käytetään tunnistamaan esimerkiksi, miten lauseen loppu liittyy sen alkuun tai miten lauseet kappaleessa liittyvät toisiinsa. Kielimallit voivat siten tulkita luonnollista kieltä jopa silloin, kun kieli on epämääräistä tai järjestetty tavalla, jota malli ei ole vielä nähnyt koulutuksessaan. [13.]

Kielimalleja koulutetaan syöttämällä niille valtavia määriä tekstidataa. Koulutukseen kuuluu miljardien sanojen prosessointi lähteistä kuten kirjoista, artikkeleista ja nettisivuista. Syntyneen tietoaineiston (engl. data set) monimuotoisuus ja riittävä koko on kriittistä, jotta kielimalleilla on hyvä oppimispohja. [3.] Koska

mallin tuotokset vastaavat sitä, mitä sinne on koulutusvaiheessa syötetty, on erityisen tärkeää käyttää korkealaatuista ja käyttötarkoitukseen relevanttia dataa. Jos datassa on virheitä, alkaa niitä esiintymään vastauksissakin.

Koulutuksen ensimmäinen vaihe on ihmisen ohjauksen alaista. Ihminen syöttää kielimallille keskusteluita ja muita tekstejä. Toisessa vaiheessa käytetään vahvistusoppimista ja valvovan ihmisen palautetta mallin vastausten sijoittelussa paremmuusjärjestykseen. Mallia koulutetaan tällä tavoin monia iteraatioita. Samalla mukaan lisätään valvomatonta oppimista, jossa malli oppii tunnistamaan kaavoja ja yhteyksiä ilman erillistä ohjausta. Lopputuloksena saadaan niin sanottu perusmalli (engl. foundation model), jota voidaan käyttää halutessaan jo sellaisenaan. [3.]

Geneerisen koulutuksen jälkeen kielimalleja voidaan vielä hienosäätää käyttötarkoitukseensa paremmaksi. Tämä lisäkoulutus tapahtuu antamalla mallille materiaalia, joka liittyy esimerkiksi tiettyyn alaan tai jopa yritykseen. [3.] Yritys, joka valmistaa esimerkiksi lääketieteellistä teknologiaa saisi siis osuvampia vastauksia kielimallilta, jos se hienosäätää sitä omiin tuotteisiinsa ja alaansa liittyvällä dokumentaatiolla ja muulla datalla. Ilman tätä koulutusta malli ei välttämättä osaisi vastata oikein kysymyksiin kyseisen yrityksen tuotteista. Hienosäätö ei kuitenkaan ole kaikissa tapauksissa pakollista, vaan se riippuu mallin käyttötapauksesta.

Kielimallien kyvyt eivät rajoitu pelkästään tekstintuottoon, vaikka se onkin niiden perinteinen ja yleisin käyttötarkoitus. On olemassa myös multimodaalisia malleja, jotka on koulutettu prosessoimaan dataa useista lähteistä. Nämä lähteet voivat olla esimerkiksi tekstiä, kuvia, ääntä ja videoita. Kuvassa 1 esiteltiin malleja, jotka kykenevät ottamaan vastaan vain tekstimuotoisia syötteitä. Sen sijaan multimodaaliset mallit, riippuen käyttötarkoituksesta, voivat esimerkiksi saada syötteenä tekstin lisäksi kuvan ja kertoa, mitä siinä näkyy tai saada tekstinä kuvauksen ja tuottaa käyttäjälle sen mukaisen kuvan. Multimodaaliset mallit saavat tekoälyn tuntumaan ihmiselle luonnollisemmalta, sillä niillä on tavallaan useampia aisteja, eikä vain kyky lukea tekstiä. Useiden lähteiden ansiosta ne

myös vastaavat käyttäjälleen tarkemmin ja ymmärtävät kontekstia monipuolisemmin. [14.]

Kaksi yleistä tyyppiä kielimalleille ovat niin sanotut Chat- ja Instruct-mallit. Kuuluisin esimerkki chat-tyyppisestä kielimallista on OpenAI:n ChatGPT-sovelluksen tällä hetkellä käyttämä GPT4o. Tämän tyyppiset mallit on koulutettu ja hienosäädetty tuottamaan keskusteluita käyttäjän kanssa ja siten sopivat käytettäväksi esimerkiksi chattiboteissa. Instruct-mallit sen sijaan on luotu seuraamaan ohjeistusta mahdollisimman tarkasti tuottamatta ylimääräistä tekstiä. [15.]

2.2.1 Syötteet

Kielimalleja käytetään antamalle niille tekstimuotoisia luonnollisen kielen syötteitä, mikä tekee malleista monipuolisia, helppokäyttöisiä sekä matalalla vaivalla mukautettavia hyvin erilaisiin tehtäviin. Pienikin muokkaus syötteeseen voi muuttaa lopputulosta suuresti, joten oikeanlaisen syötteen löytäminen vaatii pitkälti kokeilemista. Syötteet toimivat siis ohjeena mallille siitä, mitä sen kuuluu tuottaa lopputuloksena. [16.]

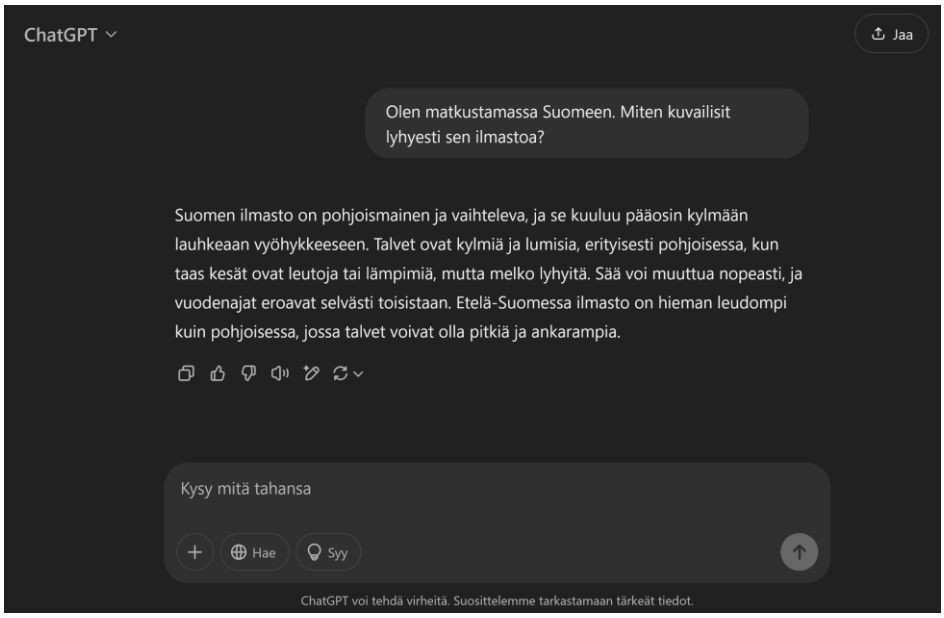
Syötteiden monimutkaisuus kasvaa sitä mukaa, mitä tarkempia tai yhtenäisempiä lopputulosten täytyy olla. Yksinkertaisella käskyllä, kuten ”Kerro sankaritarina”, saadaan hyvin erilaisia tarinoita. Ennen varsinaista käskyä mallille voidaan asettaa ohjeita tarinan kululle, kuten ”Sankarin täytyy saada legendaarinen miekka. Sankarin täytyy kohdata vanhoja ystäviä.”. Ohjeet lisäämällä mallin pitäisi tuottaa sopiva tarina, mutta täyttä varmuutta ehtojen täyttymiselle ei voi saada.

Käyttötarkoitukseen olennaisesti kuuluvat ohjeet annetaan kielimallille systeemin syötteenä (engl. system prompt). Tällaiset syötteet pysyvät yleensä muuttumattomina ja annetaan mallille joka kerta, kun sitä käytetään. Nämä ohjeet antavat mallille suuntaa siitä, miten sen pitäisi vastata käyttäjän omiin syötteisiin. Esimerkiksi chattibotissa käytettävälle mallille annettavat systeemin syötteet voisivat olla seuraavanlaiset:

- Olet ystävällinen tekoälyavustaja, joka vastaa käyttäjän antamiin kysymyksiin.
- Käytä asiantuntijamaista ja virallista kieltä.
- Kieltäydy vastaamasta, jos se on eettisesti sopimatonta.

Kielimallille annetaan ensin rooli, joka sen pitää täyttää. Seuraavaksi mallille annetaan kielelliset ohjeet ja viimeiseksi toimintaohje siltä varalta, että vastaan tulee tietynlainen tilanne. Systeemin syötteiden on tarkoitus toimia vahvempina ohjeina kielimallille kuin käyttäjän syötteet. [17.]

Kuvassa 2 näkyy tyypillinen chattibottisovellus, joka käyttää taustalla kielimallia tuottamaan vastauksia. Ylhäällä näkyy käyttäjän luoma syöte, johon kielimalli on sitä alempana vastannut. Kuten kuvasta huomaa, systeemin syötettä ei näytetä ollenkaan.



Kuva 2. Kuvakaappaus ChatGPT:stä, jossa näkyy käyttäjän syöte sekä kielimallin vastaus.

Systeemin syötteet saattavat joissain tapauksissa sisältää sensitiivistä tietoa ja yrityssalaisuuksia, mikä johtaa sovelluksen kehittäjiä pyrkimään piilottamaan kyseinen syöte käyttäjiltä. Mikäli yritys lisää salaista dataa systeemin syötteeseen, sen on syytä myös tehdä toimenpiteitä syötteen paljastumisen estämiseksi.

Oikeanlaisella syötteellä käyttäjä voi mahdollisesti saada kielimallin kertomaan, mikä systeemin syöte on käytössä, ellei sitä suojata. [18.] Sovelluksien toimintaa on muutenkin hyvä testata erilaisilla syötteillä, että voidaan varmistaa sen tuottavan oikeanlaisia vastauksia.

Hyviin syötteisiin panostaminen on tärkeää, jos haluaa myös hyviä tuloksia. Siksi syötesuunnittelusta (engl. prompt engineering) on tullut hyödyllinen taito kielimalleja käsitellessä. Tärkeää on etenkin pitää syötteet selkeinä ja yksityiskohtaisina, jotta niille saadaan relevantteja vastauksia. Mallia voidaan ohjeistaa parempiin vastauksiin esimerkiksi käskemällä sitä esiintymään jossain roolissa, ohjeistamalla sille haluttu vastauksen pituus, erottelemalla selkeästi konteksti kysymyksestä tai antamalla esimerkkejä halutusta vastauksesta ennen varsinaista kysymystä. Erilaisia syötteitä kannattaa testata systemaattisesti, sillä muutoksilla voi olla suuriakin vaikutuksia, mutta niitä voi myös olla vaikea havaita. [17.]

Esimerkkien antaminen syötteessä on edistyneempi tekniikka, jota kutsutaan Few-shot-syötteeksi. Ideana on antaa ensin kontekstina muutama kysymys-vastaus-pari, joka näyttää kielimallille, miten sen kuuluu vastata edellisiin kysymyksiin. Syöte loppuu kysymykseen, johon kielimallin kuuluu vastata. Tekniikan haittapuolena on, että se saattaa lisätä syötteiden pituutta roimasti sekä malli saattaa löytää kontekstista kaavoja ja sääntöjä, joita sen ei tarkoitettu oppivan. Syötettä, johon ei lisätä esimerkkejä ollenkaan, kutsutaan Zero-shot-syötteeksi. Tällaisesta syötteestä hyvin selviytyminen on yksi tärkeä mittari kielimallin suorituskyvylle. [16; 17.]

2.2.2 Rajoitteet

Kielimallit osaavat vain niitä kieliä, joita löytyy niiden koulutuksessa käytetystä datasta. Usein kielimallit, joiden pääkieli on jokin muu kuin englanti, käyttävät silti valtavasti englanninkielistä dataa. Suuremman kielialueen data mahdollistaa mallille laajemman yleistietämyksen, sillä koulutusdata on helpommin saatavilla. Toisaalta se voi myös johtaa kielimallin monimuotoisuuden kaventumiseen

ja siihen, että mallin ajattelu pohjautuu muiden kielten teksteihin. Tässä vaiheessa mallilta voi kadota tärkeää kulttuurisidonnaista ymmärtämistä. [19.] Sovelluksissa kielirajoite voidaan kiertää käyttämällä käännösohjelmaa käyttäjän ja kielimallin välillä, vaikka se ei poistakaan kaikkia ongelmia.

Koulutusdatan sisällön päiväykset määrittävät, mihin ajanjaksoon kielimallin tieto sijoittuu. Malleilta siis löytyy aina rajakohta (engl. knowledge cut-off date). Kielimallilta ei kannata kysyä ajankohtaisia tietoja, jos sen koulutuksesta on pitkä aika ja käyttäjä epäilee, että kysyttävä tieto on voinut jo muuttua ajan kuluessa. Kielimallien kehittäjät usein kertovat, mikä tiedon rajakohta mallilta löytyy, mutta ilmoitettu ajankohta ei kerro varmasti, miten uutta tieto on. Kehittäjien käyttämä data voi olla vanhentunutta jo koulutusvaiheessa. [20.] Ajankohtaisempaa tietoa voidaan saada kielimallin ulkopuolelta esimerkiksi syöttämällä sille sitä kontekstina kysymyksen kanssa, mutta tällöin käyttäjällä on myös oltava kyseinen uusi tieto. Sovelluksissa voidaan antaa kielimallille kyky etsiä tietoa internetistä, mikä ei anna varmuutta tiedon oikeellisuudesta, mutta kiertää tarpeen etsiä sitä itse.

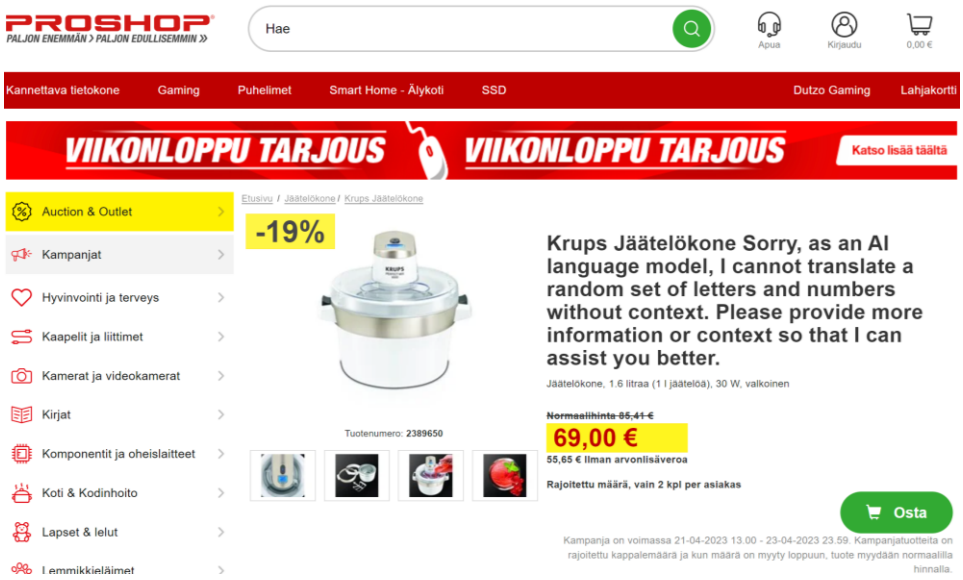
Syötteiden kokoa rajoittaa kielimallin konteksti-ikkuna eli kontekstikoko, joka on joka mallilla erilainen. Kontekstikoko tarkoittaa syötteen maksimikokoa, joka mitataan sanojen sijaan merkitysyksikköinä (engl. token). Merkitysyksiköllä voi koostua lauseesta, yhdestä sanasta, sanan osasta tai pelkästä merkistä. Olennaista on se, mikä merkitys merkitysyksiköllä on. Esimerkiksi sana ”koirat” voidaan pilkkoa merkitysyksiköihin ”koira” ja ”t”, sillä t-merkki sanan lopussa muuttaa sen monikoksi. Kontekstikoko koostuu sekä mahdollisesta systeemin syötteestä ja käyttäjän syötteestä että kielimallin generoimasta tekstistä. [21.] Koko määrittää siis, kuinka paljon tekstiä mallille voi syöttää, mikä voi olla ongelmallista käyttäjän syötteelle. Pitkät kontekstit vaativissa sovelluksissa on keksittävä, miten kielimallille annetaan vain tärkein ja relevantein tieto.

Pitkissä konteksteissa ongelmana on myös se, että ne lisäävät valtavasti vaadittavaa laskentatehoa. Jos syötteen merkitysyksiköiden määrä kaksinkertaistuu, malli tarvitsee neljä kertaa enemmän tehoa. Mallin generointinopeus myös

hidastuu sitä mukaa, mitä enemmän kontekstia sen on käsiteltävä. Tämä haittaa käyttötapauksia, joissa vaaditaan pikaisia vastauksia. Tutkimukset ovat osoittaneet, että kielimallit suoriutuvat parhaiten, kun relevantein data kontekstissa löytyy joko alusta tai lopusta. Uudet tekniikat ovat kuitenkin parantaneet mallien suorituskykyä myös relevantin datan ollessa vaikeaa löytää pitkästä tekstistä. [21.]

Kielimallit ovat tehokkaita tuottamaan vastauksia, mutta esimerkiksi koodaustehtävissä ne eivät vielä pärjää ohjelmistoinsinööreille. OpenAI:n teettämän selvityksen mukaan eräät markkinoiden edistyneimmistä kielimalleista eivät suoriutuneet hyvin, kun niille annettiin joukko ohjelmoijille tyypillisiä tehtäviä ja ongelmia. Vaikka kielimallit olivat nopeita tehtävissään, niiden ratkaisut olivat puutteellisia tai virheellisiä eivätkä ne osanneet metsästää bugeja pitkälle ja siten ymmärtää ongelmien juurisyitä. Tilanne voi kuitenkin kehittyä nopeasti niin, että yksinkertaiset ohjelmointitehtävät onnistuvat myös kielimalleilta. [22.] Nykyään kielimalleja voidaan jo kuitenkin käyttää koodiapureina ohjelmoijien rinnalla.

Yleinen haaste kielimallien kanssa kehittäessä on ylimääräisen tekstin tuottamisen välttäminen [23]. Vaikka käyttäjä lisäisi paljon ohjeita syötteeseen ohjaamaan mallin toimintaa, se voi silti jättää nämä huomioimatta tai odottamattomassa tilanteessa toimia eri tavalla kuin on tarkoitus. Hyvänä esimerkkinä toimii kuvan 3 (s. 12) verkkokauppa, joka on käyttänyt tuotetietojen käännöksissä kielimallia. Tuotteen nimessä lukee oikein sen merkki ja tarkoitus, mutta sen jälkeen kielimalli on lisännyt ylimääräistä tekstiä. Tekstistä voidaan päätellä, että kielimalli on saanut syötteessä kirjaimia ja numeroja sisältävän mallinumeron, jota se ei ole kyennyt kääntämään. Odottamattomassa tilanteessa malli on siis pahoitellut toimintaansa sen sijaan, että olisi vain jättänyt mallinumeron kääntämättä. Verkkokaupan kehittäjä ei mahdollisesti ole tajunnut, että näin voi käydä. Esimerkki myös osoittaa, miten kielimallien käyttämistä kannattaa välttää etenkin kuluttajien nähtävillä, jos ei ole varmuutta sen toimimisesta oikein kaikissa tilanteissa.



Kuva 3. Kuvakaappaus verkkokaupasta, jonka tuotteen nimeen on päätynyt ylimääräistä tekstiä kielimallin käyttämisen vuoksi.

Kielimalleilla on tapana tuottaa hallusinaatioita eli väärää tietoa, kun ne eivät kykene tuottamaan oikeita vastauksia. Samoin kuin väärä tieto koulutusdatassa johtaa vääriin vastauksiin, myös puuttuva data johtaa niihin. Kielimallilta voidaan esimerkiksi pyytää uutisartikkelia, ja se helposti luo sellaisen, mutta artikkelin faktat saattavat olla täysin tekaistuja. [13.] Suuri ongelma tässä on kielimallin vakuuttavuus. Malli saattaa tuottaa todelta kuulostavaa tekstiä, jos se ei koulutuksessa ole saanut aiheesta valmiita faktoja. Tuotettu teksti täten tarvitsee faktantarkistusta, jos malli ei ole saanut faktoja syötteen kontekstista. Käyttäjät saattavat kuitenkin uskoa tuotettua tekstiä, koska kielimalli puhuu itsevarmasti, vaikka sen väitteillä ei ole minkäänlaista pohjaa. Mallit saattavat myös yksinkertaisesti tulkita annettua dataa väärin ja näin tuottaa hallusinaatioita, vaikka datassa itsessään ei olisi mitään virheitä [3].

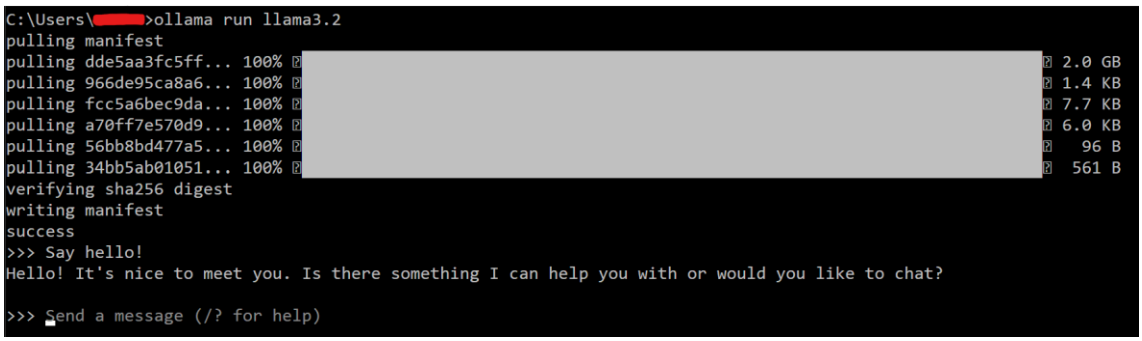
Kielimallin koko vaikuttaa sen tuottamien vastausten laatuun. Suuremmat mallit ovat yleensä suorituskykyisempiä ja tarkempia vastauksissaan kuin pienemmät, mutta samalla järjestelmävaatimusten taso nousee korkeammaksi. Parhaimmat mallit on suunniteltu datakeskuksissa käytettäväksi ja voivat siten olla kymmenien tai satojen gigatavujen kokoisia. Kielimallien ajaminen näytönohjaimella ja VRAM:illa eli sen omalla muistilla on nopeampaa kuin pelkästään tietokoneen

prosessorilla ja RAM:illa eli keskusmuistilla. [24.] Käyttäjän on tehtävä kompromissi käytössä olevien resurssien ja kielimallin suorituskyvyn välillä, sillä parhaita malleja on mahdotonta ajaa ilman suurta investointia laitteistoon.

2.3 Ollama

Ollama on alun perin vuonna 2023 julkaistu avoimen lähdekoodin sovellus, jonka tarkoituksena on helpottaa kielimallien ajamista omalla tietokoneella. Ollama toimii alustana, jolla yksinkertaisimmillaan pääsee keskustelemaan asennuksen jälkeen yhdellä komennolla, joka lataa käyttäjän määrittämän mallin sekä käynnistää sovelluksen. Ollama on saatavilla kaikille kolmelle yleisimmälle työpöytäkäyttöjärjestelmälle, jotka ovat Windows, Linux ja macOS. Myös multimodaaliset mallit sekä Embedding-mallit ovat tuettuja. [25.] Sovelluksen etuna on se, miten vaivattomasti sen saa toimimaan eri alustoilla. Käyttäjällä ei tarvitse olla suurta teknistä tietämystä kielimallien ajamisesta, vaan riittää, että hän kykenee syöttämään komentoja komentokehotteeseen.

Asennuksen jälkeen sovellusta ohjataan komentokehotteesta komennolla ”ollama”. Kuvassa 4 näytetään, miten yhdellä käskyllä päästään keskustelemaan halutun mallin kanssa. Itse kielimallin lisäksi ladataan sen lisenssi, ajossa käytettävät parametrit ja kustomoitu syötepohja, jonka avulla Ollama työskentelee mallin kanssa. Syötepohja mahdollistaa Ollamalle useiden mallien modulaarisen käytön.



Kuva 4. Komentokehote, johon on syötetty Ollaman käynnistävä käsky ja pyyntö tervehtiä.

Ollama valitsee käyttöympäristönsä perusteella automaattisesti parhaat asetukset, joita käyttää kielimallin ajoon. Esimerkiksi näytönohjainkiihdytystä käytetään ensisijaisesti sen mahdollistaman nopeuden vuoksi ja silloinkin vain, jos käytössä on yhteensopiva laitteisto. Jos käyttäjän valitsema malli ei mahdu kokonaan VRAM:iin, Ollama pyrkii jakamaan osan mallista näytönohjaimelle ja loput se jakaa RAM:iin prosessorin suoritettavaksi. Osittain käyttämällä näytönohjainta saadaan nopeushyötyjä verrattuna siihen, jos suorittamiseen käytettäisiin pelkästään prosessoria. [25.]

Sovelluksen saa myös toimimaan palvelimena, jolloin sen REST-tyylillä toteutettuun API:iin eli rajapintaan voidaan lähettää generointipyyntöjä internetin välityksellä [25]. Tämä helpottaa Ollaman integrointia sovelluksiin, jotka hyödyntävät pilvessä toimivia kielimalleja kuten OpenAI:n tuotteet. Suuri määrä erilaisia työkaluja ja ohjelmia onkin integroinut Ollaman tällä tavoin käytettäväkseen, joista työn kannalta tärkeimpänä voidaan pitää LangChainia.

2.4 LangChain

LangChain on kielimallisovellusten kehittämiseen suunniteltu avoimen lähdekoodin ohjelmistokehys (engl. framework), jonka tehtävänä on yksinkertaistaa yleisten ominaisuuksien luomista sovelluksiin. LangChain toimii geneerisenä rajapintana kielimalleille, ja sen modulaarisuus mahdollistaa kehittäjille erilaisten syötteiden ja mallien nopean testauksen vain pienillä koodimuutoksilla. Tämän ansiosta sitä voidaan käyttää hienostuneissa sovelluksissa, joissa hyödynnetään useaa kielimallia. Näin tehdään esimerkiksi sovelluksissa, joissa yhtä mallia käytetään syötteiden tulkintaan ja toista vastausten tuottamiseen. LangChain sopii hyväksi pohjaksi useimpiin kielimallien käyttötapauksiin, kuten kysymysten vastaukseen, tiivistelmien luontiin ja chattibotteihin. [26.]

Pohjimmiltaan LangChain on Pythonille ja JavaScriptille saatavilla oleva kirjasto, joka sisältää kielimallien kanssa työskentelyyn tarvittavia valmiita funktiota ja luokkia. Niitä voidaan yhdistellä minimoiden tarvittavan koodin määrän ja matalan tason ymmärryksen, jota vaaditaan monimutkaisten tekoälysovelluksien

luontiin. LangChain tarjoaa LLM-luokkia useiden kielimallien käsittelyyn, olivat ne API:n kautta käytettäviä pilvessä pyöriviä malleja tai lokaaleja malleja. LLM-luokkia ei ole rajoitettu vain kirjaston tarjoamiin valmiisiin ratkaisuihin, vaan tarjolla on myös kehittäjien räätälöitäväksi tehty CustomLLM-luokka. [26.]

LangChainista löytyy useita datan käsittelyä helpottavia luokkia. Datan lataamiseksi sovellukseen tarjotaan useita eri lähteitä käyttäviä dokumenttilataajia, jotka kykenevät tuomaan dataa esimerkiksi verkkotallennuspalveluista, verkkosivustoilta, tietokannoista tai PDF-tiedostoista. Suuret latauksessa syntyneet tekstidokumentit kannattaa käsittelynopeuden vuoksi pilkkoa pienempiin osiin, minkä vuoksi LangChainista löytyy text splitter -luokkia. Yritys voisi esimerkiksi tuoda PDF-tiedostosta tuotteensa käyttöohjeet ja pilkkoa ne sivun kokoisiin palasiin. Lisäksi LangChain tarjoaa työkaluja sekä chattisovellusten keskustelumuistia että vektoritietokantojen käyttöä varten. [26.]

2.5 Vektoritietokannat ja vektorihaku

Vektoritietokanta tallentaa, käsittelee ja indeksoi vektoridataa. Tällaisessa tietokannassa tieto tallennetaan numeroista koostuviin taulukoihin (engl. array), joita kutsutaan vektoreiksi ja jotka on ryhmitelty samankaltaisuuden perusteella. Vektorien käytön vuoksi vektoritietokannat sopivat erityisesti ei-rakenteellisen tiedon käsittelyyn. Vektoritietokantojen suosio on kasvanut, koska ne tarjoavat tekoälysovellusten kaipaamaa nopeutta niiden käyttötarkoituksiin. [27.] Toisin sanoen vektoritietokannat toimivat sovelluksen numeropohjaisena muistina, josta haetaan sinne tallennettu relevantti tieto kysymyksen perusteella vektorihakuna.

Vektorihaku tarkoittaa tiedonhakua, jossa etsitään haetun vektorin kanssa semanttisesti samankaltaista vektoria vektoritietokannasta. Tekstin muuttamisen vektoreiksi suorittaa siihen erikoistunut kielimalli, joka on koulutettu tunnistamaan samankaltaisuuksia. Malli koulutetaan muuttamaan samankaltaiset tekstinpätkät samankaltaisiksi vektoreiksi. Näin syntyneitä vektoreita kutsutaan termillä ”text embedding”. [28.]

Havainnollistavana esimerkkinä vektoreista ja niiden etäisyyksistä toisistaan esitellään neljä sanaa: ”apple”, ”orange”, ”dog” ja ”iPhone”. Embedding-malli käy sanat läpi, tulkitsee niiden merkityksen ja muuntaa ne lopulta vektoreiksi. Vektoreiden ”apple” ja ”orange” välinen etäisyys on pienempi kuin niiden etäisyys vektoriin ”dog”. Vektorihaussa siis syötetään esimerkiksi tekstinpätkä kuten kysymys hedelmistä, jonka pohjalta luodaan vektoreita ja etsitään tietokannasta samankaltaisuuden perusteella sopivia vektoreita. Näin tietokannasta pitäisi palautua hedelmiin liittyvät vektorit. Embedding-malli voi kuitenkin yllättää tekemällä vektoreista ”apple” ja ”iPhone” samankaltaisempia kuin muista vektoreista. Malli on tässä tapauksessa löytänyt näiden sanojen välillä yhteyden, jota se pitää vahvempana kuin sanojen ”apple” ja ”orange” yhteyttä.

Vektorihaun vahvuutena voidaan pitää kielimallien kontekstinymmärryskykyä. Otetaan esimerkkinä PDF-dokumentti, joka sisältää ravintolan ruokalistan ja jonka ateriat on muunnettu vektoreiksi. Tavallisessa haussa suoraan dokumentista hakusanalla kuten ”hedelmä” ei välttämättä löydy ollenkaan tuloksia. Sen sijaan vektorihaussa kysymällä ”Missä aterioissa on hedelmiä?” löydetään useita tuloksia.

2.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) on menetelmä, jolla nimensä mukaisesti noudetaan relevanttia, kielimallin ulkopuolista tietoa kielimallin syötteisiin hyödyntäen sitä vastauksen generoinnissa. Aluksi tiedonlähteenä toimiva teksti pilkotaan sopiviin osiin, muunnetaan vektoreiksi ja tallennetaan vektoritietokantaan. Kun RAG:ia käyttävässä kielimallisovelluksessa käyttäjä kysyy kysymyksen, se myös muutetaan vektoreiksi vektorihakua varten. Haun lopputuloksena saadut tulokset sisällytetään kielimallin syötteeseen kontekstina alkuperäiselle kysymykselle. Lähestymistavan toimiminen edellyttää, että kysymys ja tiedonlähde ovat semanttisesti samankaltaisia ja vektoriksi muuttava kielimalli osaa tunnistaa niiden keskisen samankaltaisuuden. [28.]

RAG:in tarkoitus ja hyöty vastauksen generoinnissa on se, että RAG mahdollistaa sovellusten käyttävän vastauksissaan ajankohtaista tietoa parantaen vastausten tarkkuutta ja luotettavuutta [28]. RAG on yksi ratkaisu aikaisemmin mainittuun kielimallien kontekstikokorajoitteeseen. Kontekstikoko voidaan mieltää mallin lyhyeksi työmuistiksi, jolloin sovelluksessa RAG:in tietolähde olisi sen pitkä muisti, josta tarvittaessa noudetaan relevantti tieto. RAG:in etuna on se, että kielimallille ei tarvitse kouluttaa tai hienosäätää kaikkea tietoa, vaan se kompensoi mallien tiedonpuutetta antamalla niille olennaista tietoa vain, kun sitä tarvitaan. Ajankohtaista tietoa vaativissa tapauksissa kysymysten syöttäminen suoraan kielimalleille on epäluotettavaa, sillä tiedon rajakohdat ja hallusinaatiot voivat vaikuttaa kielimallien vastauksiin. Kun kontekstina saadaan varmaa ja relevanttia tietoa, vastauksiin voidaan luottaa enemmän.

RAG-menetelmää hyödyntävää sovellusta voidaan optimoida monin tavoin, kuten esimerkiksi esikäsittelemällä lähdetekstit kunnolla. Pitkän dokumentin käytön sijaan teksti tulisi pilkkoa pienempiin osiin niin, että muodostuvat kokonaisuudet ovat semanttisesti järkeviä. Vektorit edustavat tällä tavalla tekstiä paljon paremmin ja vektorihaku onnistuu todennäköisemmin. Itse tekstiäkin voidaan tiivistää tai uudelleenjärjestää järkevämpään muotoon. Myös kysymysten esikäsittely voi parantaa tuloksia muotoilemalla kysymykset tarkemmin tai täydentämällä niitä laajemmalla kontekstilla. [28.]

Toinen tapa optimoida on hakutulosten uudelleenjärjestely, sillä relevantteja hakutuloksia voi tulla suuri määrä. Yleensä hakutuloksia palautetaan vain tietty määrä ja tällöin kaikista paras tulos voi jäädä pois. Yhtenä ratkaisuna RAG-toteutukseen voidaan lisätä tiedonhaun ja syötteen muodostamisen väliin reranker-kielimalli, jonka tehtävänä on järjestää ja priorisoida hakutulokset paremmin kuin, mihin Embedding-malli on kyennyt. [28.]

3 Lokaalit kielimallit

Lokaalit tai paikalliset kielimallit tarkoittavat käyttäjän näkökulmasta sellaisia kielimalleja, joita voidaan ajaa omalla koneella tai palvelimella. Lokaaleja

kielimalleja voidaan siis käyttää ilman verkkoyhteyttä. Käytännössä mikä tahansa netistä ladattava avoin kielimalli voi olla lokaalisti ajettava, mutta kuluttaja- ja yritystietokoneiden laitteisto rajoittaa, mitä malleja voidaan valita käyttöön. Tässä luvussa esitellään lokaalien kielimallien hyödyt sekä rajoitteet ja esitellään, millä perusteilla insinöörityössä käytettävä malli on valittu.

3.1 Hyödyt

Lokaalien kielimallien käytön ilmeisin hyöty on se, että kehittäjillä on suurempi kontrolli sovellusten toiminnan ja kustomoitavuuden suhteen. Pilvipohjaiset kielimallisovellukset ovat tehokkaita, mutta niiden käyttö on maksullista eivätkä ne välttämättä taivu jokaiseen käyttötarkoitukseen. Mitä erikoisemmaksi ja uniikimmaksi haluttu sovellus menee, sitä enemmän kehityksessä tarvitaan joustavuutta. [29.] Siihen päästään muista toimijoista riippumattomilla ratkaisuilla, joita voi ajaa joko pilvessä, yrityksen palvelimilla tai jopa omalla tietokoneella.

Koska kielimalleja voidaan pyörittää lokaalisti ilman verkkoyhteyttä, niitä hyödyntäviä sovelluksia voidaan käyttää myös tilanteissa, joissa pääsy verkkoon on hankalaa [29]. Esimerkiksi junamatkalla voi olla pitkiä osuuksia, joilla verkko ei toimi kunnolla ja siksi ChatGPT ei olisi käytettävissä. Omalla koneella pyörivä kielimalli mahdollistaa niiden kanssa työskentelyn myös työmatkalla.

Suurempi kontrolli kielimallista tarkoittaa myös pilvessä pyöriville malleille yleisen sensuurin vähenemistä. Alkujaan kielimallisovellukset kuten ChatGPT ja Bing AI eivät olleet kovin rajoitettuja, vaan ne kertoivat jopa pomminrakennusohjeita, jos kysymys muotoiltiin oikein. Tämän seurauksena tapahtunut mallien sensurointi johti siihen, että sovellukset saattoivat kieltäytyä kirjoittamasta väkivaltaisia kohtauksia sisältävää kirjallisuutta. Lokaalit kielimallit eivät ole täysin vapaita sensuurista, mutta joissain tapauksissa ne voivat tuottaa tekstiä, jota pilvessä toimivat sovellukset eivät suostu tuottamaan. [29.] Toinen esimerkki sensuurista on GitHubin Copilot-koodiapuri, joka ei yhdessä vaiheessa suostunut toimimaan, jos koodissa esiintyi edes näennäisesti sukupuoleen liittyviä

termejä kuten ”trans_time” [30]. Sensuuri voi siis vaikuttaa mallien suorituskykyyn heikentämällä vastausten laatua.

Sensuuri voi myös olla lähtöisin kielimalleja ja sovelluksia luovien yritysten ulkopuolelta. Kiinalaisen DeepSeek-yrityksen samannimistä chattibottia on kritisoitu sensuurista, joka kohdistuu Kiinan valtiolle epämieluisiin aiheisiin. Kielimalleja voidaan myös myrkyttää sensuurilla jo koulutusvaiheessa ohjaamalla ne vastaamaan tietyllä tavalla nähdessään avainsanan. Kielimallia valitessa on siis otettava huomioon kielimallin alkuperä ja mietittävä, onko olemassa riskiä, että yrityksellä tai siihen mahdollisesti vaikuttavalla valtiollisella toimijalla on ideologinen syy vaikuttaa kielimallin toimintaan sensuroimalla sitä. [30.]

Yksi tärkeimmistä syistä lokaalien kielimallien käyttämiseen on niiden antama varmuus, että kielimallin kanssa käydyt keskustelut ja niihin liittyvä data pysyy täysin yksityisenä. Tämä mahdollistaa myös sensitiivisen datan syöttämisen sovellukseen luottamuksellisesti. Pilvessä pyörivää kielimallia käytettäessä sitä hallinnoivalla yrityksellä on pääsy kaikkeen syötettyyn dataan. [29.]

EU:ssa toimiville yrityksille on kriittistä noudattaa tietosuojalakeja, kuten GDPR (General Data Protection Regulation), ja muita EU:n säännöksiä, sillä muuten yritykset voivat saada sakkoja, oikeudellisia seurauksia ja mainehaittoja. GDPR-lain mukaan henkilötietoja sisältävän datan on joko pysyttävä EU:n sisällä, missä GDPR on voimassa tai ulkopuolisessa maassa, joka täyttää EU:n tietosuojakriteerit. [31.] Yhdysvalloissa toimivia kielimallipalveluita käytettäessä voi olla ongelmia turvallisen datansiirron kanssa. Yhdysvaltojen nykyinen hallinto harkitsee Yhdysvaltojen ja Euroopan unionin välisestä datansiirtokehyksestä (DPF) luopumista, mikä johtaisi henkilötietojen siirron laittomuuteen [32]. Lokaaleja kielimalleja käyttämällä varmistetaan, että yritys kykenee itse huolehtimaan tietosuojasta joka vaiheessa.

Lokaalien kielimallien etuna on se, että pilvessä toimivat kielimallit ovat lähtökohtaisesti maksullisia. ChatGPT:tä voidaan käyttää OpenAI:n verkkosivun kautta ilmaiseksi, mutta siinä on käyttörajoitteita. Omaa sovellusta kehitettäessä täytyy käyttää API:a, joka voi olla kallista. [29.] Lokaaleja kielimalleja

käytettäessä vaaditaan vain alkusijoitus, jos käyttäjältä tai yritykseltä ei vielä löydy sopivaa laitteistoa.

Eräässä tutkimuksessa todettiin, että lokaalit kielimallit tuottavat 500-sivuisen tekstidokumentin huomattavasti ympäristöystävällisemmin kuin ihminen vastaavassa työssä. Tutkijat käyttivät suurempaa Llama 3 70B -kielimallia ja hyvin pientä Gemma 2B -kielimallia, joista pienempi oli selkeästi energiatehokkaampi kuin suurempi malli tai ihmiset. Tutkijat väittivät, että kun kielimallin koko kasvaa, se ei enää välttämättä ole ihmistä tehokkaampi. Tutkimuksessa kuitenkin huomautettiin, ettei tehtävänä olleen puhtaan tekstintuoton tuloksia voi yleistää jokaiseen käyttötapaukseen. [33.] Mallin valinnalla voi vaikuttaa sovelluksen ympäristöystävällisyyteen, sillä aina ei tarvita kaikkein tehokkaimpia malleja.

Pilvimalleista ei halutessaan tarvitse luopua kokonaan, vaan vastikään kehitettyjen hybridimenetelmien avulla niitä voi hyödyntää lokaalien mallien rinnalla. Minion-menetelmässä pilvimalli keskustelee yksittäisen lokaalin mallin kanssa, jolla on pääsy dataan, kunnes molemmat pääsevät yhteisymmärrykseen ratkaisussa. MinionS-menetelmässä pilvimalli jakaa tehtävän pieniin osiin, joita lokaalit mallit suorittavat rinnakkain. Ensimmäinen tapa laski kuluja noin 30-kertaisesti ja säilytti 87 % pilvimallin suorituskyvystä. Toinen tapa sen sijaan laski kuluja lähes 6-kertaisesti samalla säilyttäen noin 98 % pilvimallin suorituskyvystä. [34.]

3.2 Rajoitteet

Lokaalien kielimallien rajoitteista korostuu erityisesti suorituskyky. Parhaimmat mallit kaipaavat korkeatasoista laitteistoa. Kielimallien pyörittämiseen tarvitaan tehokas prosessori, paljon RAM:ia ja erillinen näytönohjain. Peruskannettavalta voidaan odottaa, että vastausten tuottaminen on todella hidasta, sillä niissä ei yleensä ole erillistä näytönohjainta, joka kiihdyttäisi laskentaa. [29.] Vaikka saisi käyttöönsä tarvittavan laitteiston, suurimmat avoimet kielimallit eivät silti välttämättä pärjää vertailuissa pilvimalleja vastaan. Pienempi koko yleensä merkitsee myös heikompia vastauksia.

Pilvessä toimivia kielimalleja päivitetään yleensä useasti, jolloin niiden tieto maailmasta on myös tuoreempaa. Lokaaleilla kielimalleilla taas tiedon rajakohta jää usein staattiseksi, sillä ne julkaistaan yleensä vain kerran. [29.] Kehittäjän tehtävänä on joko päivittää käyttöön parempia malleja sitä mukaa, kun niitä julkaistaan, tai lisätä sovelluksen syötteisiin ajankohtaista tietoa suoraan tai verkkohaulla.

3.3 Mallin valinta

Hugging Face on alan harrastajien suosima verkkosivu, jolta löytyy ilmaiseksi valtavasti erilaisia kielimalleja. Kielimallien erojen löytämiseksi Hugging Face perusti Open LLM Leaderboard -listan, johon päivitetään mallien tuloksia suorituskykytesteistä. Käytössä on yhteensä kuusi testiä, joissa tutkitaan kykyä ymmärtää tekstiä, kykyä vastata hankaliin eri tieteenalojen kysymyksiin, kykyä päätellä, matemaattisia kykyjä ja ohjeiden seurannan tarkkuutta. Listalta voi helposti valita mallin, joka sopii testien perusteella juuri omaan käyttötarkoitukseen. [35.]

Kuvassa 5 näkyy kyseinen lista, kun siitä on etsitty Metan kielimalleja. Kuvassa näkyy myös, että suuremmat 70B-kokoiset mallit johtavat selkeästi 3B-kokoista mallia. 70B-Instruct-mallien tuloksista huomaa, että uudemmat versiot ovat keskiarvoltaan parempia kuin vanhemmat.



Rank	Type	Model	Aver...	IFEval	BBH	MATH	GPQA	MUSR	MM...	CO_2 ...
40		meta -llama/Llama-3.3-70B-Instruct	44.85 %	89.98 %	56.56 %	48.34 %	10.51 %	15.57 %	48.13 %	76.56 kg
62		meta -llama/Llama-3.1-70B-Instruct	43.41 %	86.69 %	55.93 %	38.07 %	14.21 %	17.69 %	47.88 %	40.22 kg
534		meta -llama/ Meta -Llama-3-70B-Instruct	36.37 %	80.99 %	50.19 %	24.47 %	4.92 %	10.92 %	46.74 %	36.48 kg
1424		meta -llama/ Meta -Llama-3-70B	26.71 %	16.03 %	48.71 %	18.58 %	19.69 %	16.01 %	41.21 %	46.81 kg
1482		meta -llama/Llama-3.1-70B	26.20 %	16.84 %	46.40 %	18.43 %	18.34 %	16.58 %	40.60 %	13.60 kg
1768		meta -llama/Llama-3.2-3B-Instruct	24.20 %	73.93 %	24.06 %	17.67 %	3.80 %	1.37 %	24.39 %	1.93 kg

Kuva 5. Open LLM Leaderboard -lista, josta on etsitty Metan kielimalleja.

Hugging Facesta saa kielimalleja myös suoraan Ollaman kanssa käytettäväksi. Mallien täytyy olla GGUF-formaatissa eli nimen on oltava muotoa ”malli.gguf”.

GGUF-formaatin malleja löytää suodattamalla hakuja GGUF-tunnisteella. Sopivan kielimallin löydyttyä sen sivulla täytyy klikata ”Use this model” -painiketta ja sen alta valita ”ollama”, että saa oikean komennon suorittaa malli Ollamalla. [36.]

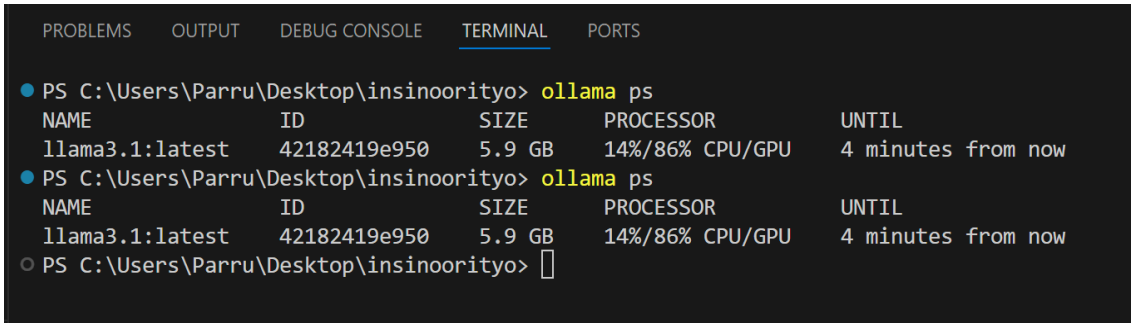
Kielimalleja selatessa löytyy vaihtoehtoja monenlaisiin käyttötarkoituksiin. Jotkut näistä malleista ovat aikaisemmin mainittuja perusmalleja, chat-malleja ja instruct-malleja. Perusmalleja voidaan käyttää sellaisenaan, mutta niitä ei ole hienosäädetty mihinkään käyttötarkoitukseen. Hienosäädön puute voi johtaa arvaamattomaan toimintaan vastauksissa. Chat-malli kannattaa valita, jos sovelluksessa on tarkoitus keskustella suoraan mallin kanssa jollain tavalla. Instruct-malli sopii sovelluksiin, joissa mallille annetaan tarkat ohjeet toiminnalle. Tällainen sovellus voi olla esimerkiksi tiivistelmien luonti tekstistä tai tarkasti määritellyn tekstinpätkän tuotto ilman ylimääräisiä merkkejä. [37.]

Kaupalliseen käyttöön menevän sovelluksen täytyy valita kielimalli sen käyttämän lisenssin perusteella, sillä niissä voi olla yrityksen toimintaa haittaavia rajoitteita. Jotkut avoimet lisenssit, kuten Apache-2.0, vaativat yrityksiltä ilmoituksen jokaisesta muutoksesta, joka kielimalliin tehdään. BSD 3-Clause -lisenssi sen sijaan ei salli kielimallin luojien mainitsemista tuotteen mainoksissa. [38.] Jotkut lisenssit ovat yritys- tai mallikohtaisia. Metan käyttämä lisenssi päivitettiin Llama 3.1 -mallin julkaisun myötä sallimaan kielimallien vastauksien käytön muiden mallien parantamiseen [39]. Käyttöehdot voivat siis kehittyä sallivampaankin suuntaan.

Kielimallin koko on yksi tärkeimmistä tekijöistä mallin valinnassa. Kuten kuvasta 5 (s. 21) huomaa, suuremmat mallit ovat yleensä parempia kuin pienemmät mallit etenkin, jos ne ovat saman malliperheen sisällä. Kielimallien koko ilmoitetaan miljardien parametrien määrällä. Parametrit ovat kielimallien koulutuksessa oppimat numeeriset arvot, joita ne säätävät ymmärtääkseen luonnollista kieltä. Mallin koko tarjoaa käytännöllisen tavan arvioida sen suorituskykyä. Kahden samankokoisen mallin voidaan arvioida olevan suurin piirtein samassa luokassa suorituskyvyltään, mutta pienemmätkin mallit saattavat olla suurempien

vertaisia tietyissä tehtävissä. Kielimalleista löytyy hyvin usein myös versioita, joihin on käytetty Quantization-tekniikkaa. Tekniikan käyttö vähentää mallin kokoa suorituskyvyn kustannuksella. Lopulta mallin valinta koon perusteella riippuu myös käyttötarkoituksesta, sillä pienempi hienosäädetty kielimalli voi toimia paremmin kuin isompi geneerinen kielimalli. [40.] Jos hienosäätöä ei tarvitse tehdä, on parasta valita suurin muistiin mahtuva malli olettaen, että se ei ole käyttötarkoitukseen liian hidas.

Käytössä oleva laitteisto määrittää, kuinka iso kielimalli voidaan ottaa käyttöön. Kielimallin tulee mahtua vähintään RAM:iin, että sitä voidaan ajaa. Vielä parempi on, jos kielimalli mahtuu kokonaan VRAM:iin, sillä silloin laskentaa voidaan kiihdyttää näytönohjaimella sitä tukevilla ohjelmilla. [25.] Kuvassa 6 näkyy, miten Ollama osittaa kielimallin prosessorin ja näytönohjaimen välillä. Käytetyllä näytönohjaimella on muistia 8 gigatavua, joka on suurempi kuin käytetyn mallin vaatima tila. Kaikkea muistia ei kuitenkaan saa Ollaman käyttöön, sillä loput muistista on muiden sovelluksen käytössä. Siksi mallista vain 86 % ajetaan näytönohjaimella. Kuvan kielimallin koko on 8B.



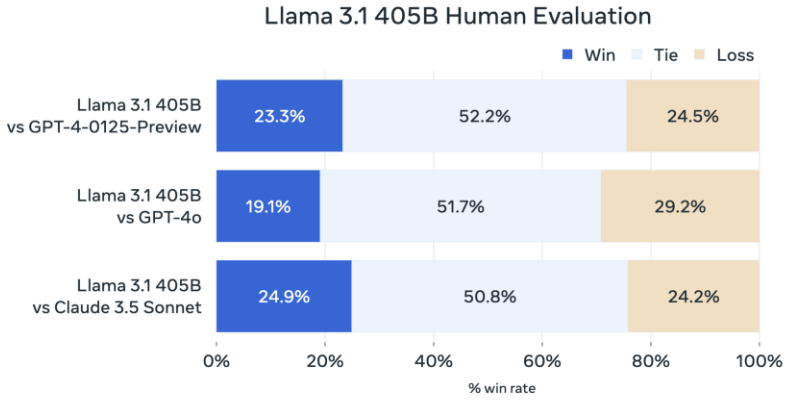
Kuva 6. Terminaali, jossa näkyy Ollamassa pyörivän kielimallin käyttämä muisti ja sen jako.

Myös Suomessa kehitetään kielimalleja. Suomalaisilla avoimilla kielimalleilla on etuna kulttuurinen yhteensopivuus, jos sovelluksen käsittelemä teksti on suomenkielistä. [19.] Niihin voi myös luottaa enemmän niiden alkuperän vuoksi. Valtioneuvoston kokeilun [41] mukaan ainakaan suomalainen Poro-kielimalli ei ainakaan ole vielä riittävän suorituskykyinen yhteenvetojen tuottamisessa

lainvalmistelijoiden lausunnoista sen pienen kontekstikoon vuoksi. Toinen kielimallin haaste on palautteen yleistys, eli se ei ymmärrä, kohdistuuko palaute nykyiseen lainsäädäntöön vai lausuntokierroksella olevaan ehdotukseen. Kokeilussa todettiin, että ollakseen hyödyksi kielimallin tulee olla kontekstikooltaan suurempi ja koulutettu käyttötarkoitukseen sopivalla materiaalilla.

Insinöörityön tilaajaksi keksitty yritys halusi selvittää, kyetäänkö lokaaleja kielimalleja ajamaan tyypillisillä työkannettavilla. Verkkokauppoja selaamalla selviää, että nykyisten työkannettavien standardikoko RAM:ille on 16 gigatavua, eikä niillä ole erillisiä näytönohjaimia. Suurempiakin vaihtoehtoja löytyy, mutta oletetaan yrityksen omien kannettavien RAM:in vastaavan tätä kokoa. Insinöörityötä varten käytössä on kuusi vuotta vanha kannettava, jonka RAM:in koko on myös 16 gigatavua. Lisäksi vertailua varten on käytössä pöytätietokone erillisellä näytönohjaimella, jolla on muistia 8 gigatavua. Kannettavan ikä ja erillisen näytönohjaimen puute tulee ottaa huomioon kielimallien suorituskykyä arvioidessa. Nämä seikat huomioon ottaen valitun kielimallin koon tulisi olla enintään 8B, jotta se mahtuisi VRAM:iin lähes kokonaan.

Kuvassa 7 esitellään ihmisvastaajien mielipiteitä vertailussa muihin Llama 3.1 -mallin julkaisun aikaisiin huipputason malleihin. Mallin koko on perheensä suurin eli 405B, mutta suurimman koon suorituskykyä voidaan pitää indikaattorina kielimallin yleiselle tasolle. Pienemmän kielimallin valinta on kuitenkin välttämätöntä laitteistorajoitusten vuoksi.



Kuva 7. Llama 3.1 405B -kielimallin vertailu muihin huippumalleihin [39].

Lopulta kielimalliksi valikoitui Metan Llama 3.1 8B, jota Ollama tarjoaa ensimmäisenä vaihtoehtona käyttäjälle. Kokonsa vuoksi se sopii hyvin käytössä olevalle laitteistolle. Metan Llama-kielimallit ovat hyvin suosittuja ja niiden vaikutus alaan näkyy monien yleisessä käytössä olevien työkalujen, kuten Ollaman ja llama.cpp:n, nimen inspiraationa. Tämän ja muiden luvussa mainittujen seikkojen vuoksi Llama-mallit vaikuttavat sopivalta pohjalta aloittaa lokaalien kielimallien käytön tutkiminen.

4 Sovelluksen toteutus

4.1 Sovelluksen määrittely

Insinöörityön tutkimuksen aiheena olevien lokaalien kielimallien hyödyllisyyden demonstroimiseksi yrityksille luotiin esimerkkiprojekti. Työn tilanneella yrityksellä on pitkiä yrityksen valmistamiin tuotteisiin liittyviä käyttöohjeita PDF-formaatissa. Asiakkailla on vaikeuksia löytää haluamaansa tietoa käyttöohjeista. Ongelman ratkaisu voi nostaa asiakastyytyväisyyttä ja vähentää yrityksen asiakaspalvelun kuormitusta. Toinen käyttötapaus on, että yrityksen työntekijät voisivat käyttää sovellusta tiedonhakuun yrityksen PDF-dokumenttien kanssa, jolloin dokumenttien käsittely tapahtuu täysin yrityksen omalla laitteistolla.

Sovellukselle asetetaan muutamia vaatimuksia, jotka sen tulee täyttää sopiakseen ratkaisuksi. Sovelluksen tulee

- olla helppokäyttöinen ja selaimella käytettävä
- vastaanottaa PDF-dokumentteja käyttäjiltä
- vastaanottaa kysymyksiä ja näyttää kielimallin vastaukset
- perustaa vastaukset dokumenttien sisältöön
- näyttää käytetyt lähteet sivunumeroina käyttäjälle
- vastata mielellään alle minuutissa ollakseen responsiivinen.

4.2 Kielimallin ajaminen

Insinöörityöprojektin toteutus aloitettiin testaamalla, toimivatko kielimallit käytössä olevalla laitteistolla. Tätä varten vaadittiin ohjelma, jolla ajetaan kielimalleja. Vaihtoehtoja tutkimalla löytyi yksinkertainen ja suosittu Ollama-ohjelma, joka ladattiin tietokoneelle kokeilua varten. Ollama tarjoaa Windowsille asennuspaketin, jonka suorituksen jälkeen ohjelma jää taustaprosessina odottamaan komentoja. Ohjelman käynnistämiseksi avattiin komentokehote ja syötettiin komento:

```
ollama run llama3.1
```

Esimerkkikoodi 1. Ollaman käynnistys valitulla kielimallilla

Komennolla Ollama käynnistyy ja asentaa komennossa määritellyn kielimallin, jos sitä ei ole vielä aikaisemmin asennettu. Mallin latauksen jälkeen Ollama siirtyy chattitilaan, jossa voidaan keskustella suoraan mallin kanssa. Kuvassa 4 (s. 13) näytetään esimerkki tilanteesta eri mallilla. Kielimalli vastaa nopeasti kysymykseen, mikä vahvistaa ohjelman toimivuuden. Seuraavaksi voitiin siirtyä kehittämään itse sovellusta, joka hyödyntää Ollamaa.

Seuraavana kokeiltiin kielimallin vastauksen generointia LangChainin kautta. LangChain valittiin sovellukseen, koska se on Ollaman tapaan yleinen työkalu kielimallien kanssa työskennellessä ja sen käyttö helpottaa kehitystä huomattavasti. Kielimallien edustamiseen löytyy useita LLM-luokkia, joista työssä

käytetään OllamaLLM-luokkaa. Esimerkkikoodissa 2 näytetään, miten yksinkertaisesti LangChainilla voidaan tehdä generointipyyntö. Ensin luotiin kielimallia edustava objekti syöttämällä OllamaLLM-luokalle argumenttina haluttu kielimalli, eli Llama 3.1. Huomioitavaa on, että Ollamalle täytyy erikseen ladata halutut kielimallit, ennen kuin niitä yritetään käyttää LangChainissa. Seuraavaksi kielimallilla generoitiin vastaus annettuun kysymykseen. Prompt-muuttuja vastaa syötettä eli tehtyä kysymystä. Myös LangChainin kautta siis saatiin onnistuneesti vastaus. Valitut teknologiat todettiin työhön sopiviksi, sillä kielimallia saatiin ajettua omalla laitteistolla.

```
model = OllamaLLM(model="llama3.1")
response_text = model.invoke(prompt)
```

Esimerkkikoodi 2. Generointipyyntö Ollamalle LangChainissa

4.3 Tiedonhaku ja generointi

Dokumentin käsittely

RAG:in toteutus aloitetaan dokumentin käsittelyllä. Esimerkkikoodissa 3 (s. 28) käyttäjän lähettämän PDF-dokumentin tiedostopolku on saatavilla funktion parametrina ”pdf_filepath”. Aluksi dokumentti ladataan LangChainin PyPDFLoaderilla sovellukseen. Seuraavaksi Langchainin TextSplitter-luokalle määritellään tavoitteeksi pilkkominen osiin, jotka ovat noin 1000:n merkin pituisia ja jotka saavat varmuuden vuoksi sisältää 80 merkin verran viereisten osien tekstiä. Näin osat sisältävät sopivasti tekstiä ollakseen yleisesti hyödyllisiä, mutta parempien lukujen valinta riippuu käytettävästä dokumentista. Lopussa funktio palauttaa dokumentin tekstin pienempinä osina.

```
def process_file(pdf_filepath):

    # Load PDF document into the app
    loader = PyPDFLoader(pdf_filepath)
    docs = loader.load()

    # Split document into smaller chunks
     text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
     chunk_overlap=80)
     chunks = text_splitter.split_documents(docs)

     return chunks
```

Esimerkkikoodi 3. Funktio PDF-dokumentin prosessoinnille pienempiin osiin.

Vektoritietokanta ja tiedonhaku

Seuraavaksi toteutettiin pilkottujen osien vektorisointi vektoritietokantaan, kysymyksen perusteella haku tietokannasta sekä lähteiden ja kielimallille lähetettävän lopullisen syötteen muodostus. Vektorisointia varten LangChainissa otettiin käyttöön Embedding-malli nimeltä FastEmbed, joka ei muiden testattujen mallien tavoin vaatinut muuta kuin paketin asennuksen komentokehotteessa. Projektin vektoritietokannaksi valittiin Chroma, koska sen asennus ja integraatio LangChainiin oli yksinkertaista ja sillä kykenee joko toimimaan kokonaan muistissa tai tallentamaan tietokannan pysyvästi levylle. Tässä työssä valittiin toimia muistipohjaisesti, koska sovellus ei tällä hetkellä vaadi pysyvää tallennusta. Chromalla esiintyi kuitenkin myös ongelmia, sillä se aluksi kaatuili ilman virheilmoitusta. Ongelma korjaantui vaihtamalla Pythonin versiosta 3.9 versioon 3.12. Esimerkkikoodissa 4 (s. 29) funktio saa parametrina tietolähteen pilkotut osat sekä käyttäjän kysymyksen. Embedding-malli sekä pilkotut osat annetaan ”from_documents”-funktiolle argumentteina, jolloin Chroma tallentaa osat vektorisoituina.

Seuraavaksi suoritetaan vektorihaku Chroman ”similarity_search”-funktiolla. Funktiolle annetaan argumentteina haettava teksti sekä palautettavien tulosten määrä. Funktio vektorisoi tekstin, etsii viisi lähintä vektoria ja palauttaa ne niiden alkuperäisissä muodoissaan.

Lopuksi vektorihaun tulosten sivunumerot dokumentista otetaan muistiin ”sources”-muuttujaan. Tulokset lisätään yhteen viivoilla erotettuun tekstiin, joka sitten

annetaan LangChainin ”from_template”-funktiolle. Funktio ottaa ennaltamääritetyn “PROMPT_TEMPLATE”-syötemallin ja lisää käsitellyt tulokset syötteen kontekstiksi sekä käyttäjän kysymyksen syötteen loppuun. Esimerkkikoodin lopussa “retrieve”-funktio palauttaa äsken muodostetun syötteen sekä lähteet.

```
def retrieve(chunks: list[Document], query_text: str):
    # Add chunks to a vector store
    embeddings = FastEmbedEmbeddings()
    vector_store = Chroma.from_documents(chunks, embeddings)

    # Search the vector store for chunks most similar to given query
    results = vector_store.similarity_search(query_text, k=5)

     # Go through results of the search, noting down used sources
     # (pages of the document) and forming a prompt for the LLM
     sources = [doc.metadata.get("page_label", None) for doc in results]
     context_text = "\n\n---\n\n".join([doc.page_content for doc in results])
     prompt_template = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
     prompt = prompt_template.format(context=context_text, ques-
     tion=query_text)

     return prompt, sources
```

Esimerkkikoodi 4. Funktio tiedonhaulle ja syötteen muodostamiselle.

Esimerkkikoodissa 5 näytetään sovelluksessa käytetty syötemalli, jonka alussa kielimallia käsketään vastaamaan pelkästään alemmaksi lisättävien hakutulosten perusteella. Lisäksi kielimallille kerrotaan, että ne ovat PDF-dokumentista, jotta se ymmärtäisi, minkälaisessa sovelluksessa sitä käytetään. Lopussa vielä käsky toistetaan, mutta huomautetaan, että konteksti on ylempänä. Mallisyötteessä myös käsketään kielimallia vastaamaan pitkästi. Ennen tämän kohdan lisäystä kielimallin vastaukset olivat melko lyhyitä, mikä osoittaa syötteissä annettujen ohjeiden voiman kielimallin vastauksissa.

```
PROMPT_TEMPLATE = """
Answer the question based only on the following excerpts from a user submitted
PDF:

{context}

---

Answer the question based on the above context in great length: {question}
"""
```

Esimerkkikoodi 5. Syötemalli, johon vektorihaun tulokset ja kysymys lisätään.

Vastauksen generointi

Viimeisenä RAG:in toteutuksessa on generointiosuus. Esimerkkikoodissa 6 funktio saa parametreinaan viime vaiheessa muodostetun syötteen sekä lähteet. Seuraavana askeleena valmis syöte annetaan kielimallille vastauksen generointia varten, mikä käsiteltiin jo aikaisemmin kielimallien ajoa testatessa. Kielimallin vastaus sekä lähteet yhdistetään lopulliseen vastaukseen, joka palautetaan funktiosta sovelluksen käyttäjälle näyttämistä varten.

```
def generate(prompt: str, sources: str):
    # Send the formed prompt to the chosen LLM being run in Ollama
    model = OllamaLLM(model="llama3.1")
    response_text = model.invoke(prompt)

    final_response = f"{response_text}\nSources: {sources}"
    return final_response
```

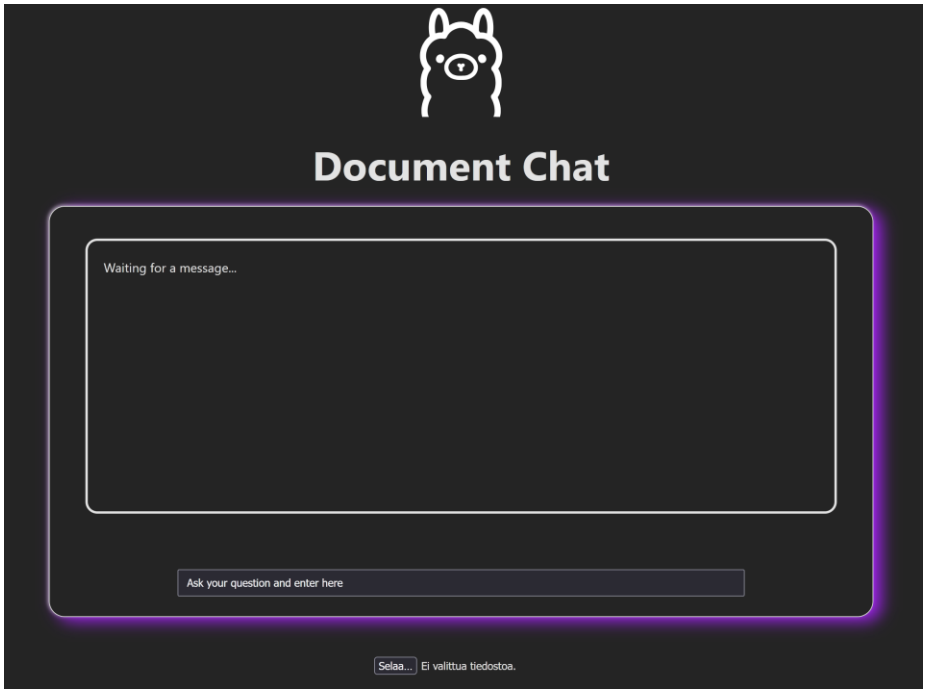
Esimerkkikoodi 6. Funktio, jolla generoidaan vastaus kielimallilla.

4.4 Palvelin ja käyttöliittymä

Sovellukselle rakennettiin seuraavaksi palvelin, joka toimii käyttöliittymän ja taustajärjestelmän välisenä rajapintana. Palvelinkirjastoksi valikoitui FastAPI [42], sillä haussa oli Pythonilla toimiva ja mahdollisimman yksinkertainen ratkaisu. Toteutus alkoi rakentamalla palvelimelle API-päätepiste PDF-dokumentin lataukselle. Päätepisteessä tarkistetaan, onko datan tallennukselle määritelty kansio olemassa, ja sellainen luodaan tarvittaessa. Palvelin myös tyhjentää kansion, koska sovelluksessa käsitellään vain yhtä tiedostoa kerrallaan. Tarkistusten jälkeen tiedosto tallennetaan palvelimelle, kunhan se on PDF-dokumentti. Toinen päätepiste ottaa vastaan käyttäjän lähettämän kysymyksen, lähettää sen PDF-dokumentin tiedostopolun mukana RAG-taustajärjestelmään ja välittää valmiin kielimallin vastauksen takaisin käyttäjälle.

Sovelluksen viimeisenä osuutena rakennettiin käyttöliittymä, jonka pohjaksi valikoitui React. Sovelluksen jatkokehitystä ajatellen React on hyvä valinta, sillä sen suosio ja komponenttipohjaisuus helpottaa kehittämistä suuresti. React ei välttämättä ollut tarpeellinen tämän tasoisen käyttöliittymän tekemiseen, mutta

nyt sovellusta on helpompi laajentaa. Kuvassa 8 näytetään sovelluksen käyttöliittymä sen alkutilanteessa. Aivan pohjalta löytyy nappi, jota painamalla voidaan valita sovellukseen syötettävä PDF-dokumentti. Napin alapuolelle ilmestyy tiedoston koko nimi, jotta dokumentin erottaa muista nimeltään samankaltaisista dokumenteista. Se myös indikoi, että tiedosto on onnistuneesti ladattu ja käytössä. Napin yläpuolella on myös laatikko, johon käyttäjän kuuluu syöttää kysymys ja painaa lopuksi ”Enter”-näppäintä. Sivu ilmoittaa kesken olevasta generoinnista ja vastaus ilmestyy keskellä olevaan tekstilaatikkoon.



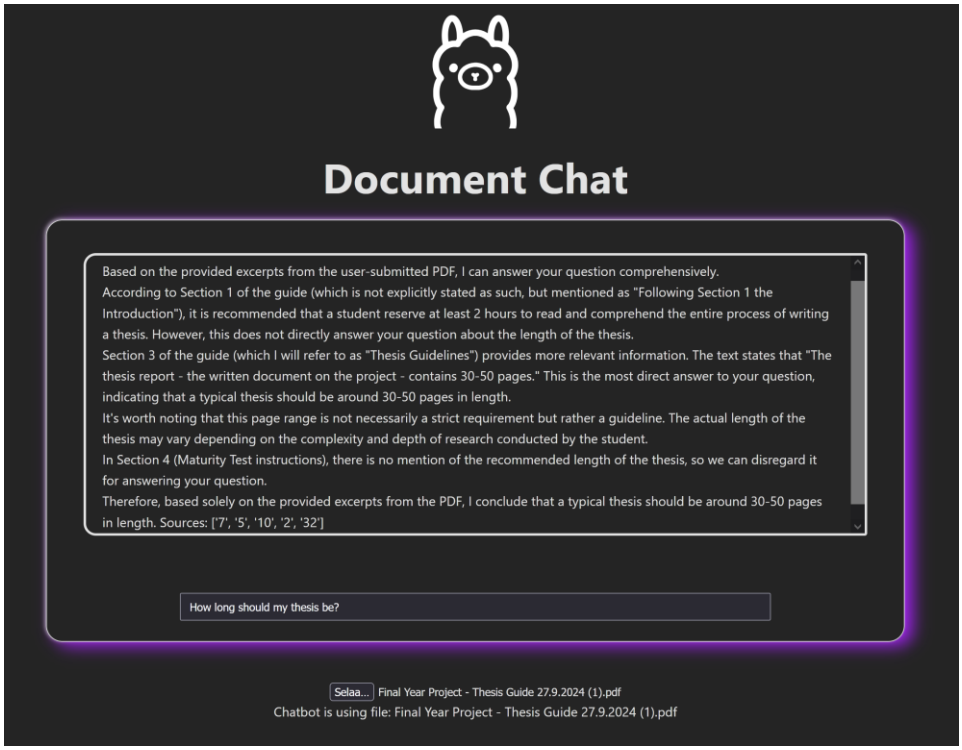
Kuva 8. Käyttöliittymä alkutilanteessa.

4.5 Sovelluksen testaus

Sovellusta testattiin käyttämällä Metropolian englanninkielistä insinöörityöohjetta käsiteltävänä PDF-dokumenttina. Dokumentilla on noin 46 sivua tekstiä. Aluksi kokeiltiin etsiä dokumentista löytyvää tietoa käyttämättä tismalleen samoja sanoja kuin miten tieto on kirjoitettu tekstissä.

Ensimmäisenä koeajona siis etsittiin Llama 3.1 8B -kielimallia hyödyntämällä tietoa insinöörityön pituusvaatimuksista kysyen ”How long should my thesis

be?”. Vastauksessa kesti pöytäkoneella noin 30 sekuntia, kun taas kannettavalla kesti noin 8 minuuttia. Kielimalli kommentoi kahta kontekstina saatua tekstinpätkää, vaikka ne eivät sisältäneet vastausta kysymykseen. RAG:illa kuitenkin löydettiin onnistuneesti relevantti tieto dokumentista. Pöytäkoneen nopeus oli kohtuullinen, mutta tavoitteiden kannalta kannettava oli liian hidas. Koeajoa jatkettiin kokeilemalla pienempiä Llama 3.2 3B- ja 1B-malleja, jolloin tuloksiksi saatiin 4 min ja 3,5 min. Vastausaika puolittui, mutta ei täyttänyt vieläkään vaatimuksia. Vastausten laatu ei kuitenkaan heikentynyt merkittävästi mallien pienentyessä. Kuvassa 9 näkyy saatu vastaus.



Kuva 9. Käyttöliittymä ensimmäisessä koeajossa vastauksen jälkeen.

Toisessa koeajossa etsittiin samaa tietoa, mutta kiertelevämmin eri kysymyksellä eli ”What should be the amount of text in my thesis?”. Tässä tapauksessa RAG ei löytänytkään etsittävää tietoa, vaan sen sijaan palautti eri osiosta tiivistelmän pituuteen liittyvää tekstiä. Vastaus ei tavallaan ollut kokonaan väärin, koska löydetty tieto kertoi tekstin määrästä.

Kolmannessa koeajossa etsittiin yleisiä ohjeita kirjoitustyölle ja kysymys oli ”What should be considered when writing a thesis?”. Lähteitä tutkimalla arvioin, että RAG onnistui löytämään hyviä kohtia dokumentista ja selitti niitä ymmärrettävästi. Kielimalli antoi pitkän vastauksen, jota toivottiinkin, eikä lisännyt täysin omia ajatuksia vastaukseen.

Lisäksi suoritettiin eri kysymyksillä monia koeajoja, jotka eivät merkittävästi eronneet aikaisemmista. Kuitenkin huomattiin, että vastauksen pituus vaikuttaa suuresti generoinnin kestoon. Pöytäkoneella Llama 3.1 8B-mallia käyttäessä vastauksissa kesti noin 0,5–1 minuuttia. Pöytäkoneen näytönohjaimen mahdollistama ero laskentatehossa oli merkittävin tekijä koelaitteiden välillä, vaikka prosessorinkin teho oli yksi tekijä. Sovelluksen vaatimukset täyttyivät kokonaan pöytäkoneella.

Merkittävä koeajoissa huomattu oikku tapahtui, kun kolmatta kysymystä testattiin monta kertaa peräkkäin. Eräällä ajolla kielimalli kieltäytyi vastaamasta vedoten akateemiseen etiikkaan. Muilla kerroilla se ei osoittanut mitään vastustusta kysymyksen vastaamisessa. Toinen merkittävä oikku tapahtui, kun kontekstina saatu teksti sisälsi mainintoja ChatGPT:stä. Kielimalli luuli, että teksti oli sen vuoksi ChatGPT:n kirjoittamaa.

5 Tulokset ja jatkokehitys

Insinöörityön tavoitteena oli saada aikaan yritykselle hyödyllinen prototyyppisovellus, jota voitaisiin ajaa yrityksen työkannettavilla. Tutkimusosuudessa löydettiin monipuolisesti tietoa hyödyistä ja rajoitteista, joita kielimalleilta löytyy. Insinöörityöprojektista kehittyi sovellus, joka vaatimusten näkökulmasta oli onnistunut. Puutteelliseksi vaatimuksista jäi vain sovelluksen suoritusnopeus käytössä olevalla laitteistolla. Pöytäkoneen koeajot osoittivat, että sovelluksen saa toimimaan suhteellisen hyvällä suoritusnopeudella, mutta kannettavalla nopeus oli iso ongelma. Yrityksen kannettavien tehoista ei ollut tietoa, joten insinöörityössä niiden suorituskyvyn oletetaan olevan todennäköisesti jossain kahden testatun

laitteen välillä. Asia voi muuttua, jos yrityksen kannettavat ovat noin vuoden sisällä ostettuja.

Harvoilla yrityskannettavilla on erillinen näytönohjain, mutta niiltä kuitenkin löytyy integroitu näytönohjain. Verkkokauppojen tarjontaa tutkimalla uusista yrityskannettavista löytyy esimerkiksi vuoden 2023 lopussa julkaistun Intelin Core Ultra -sarjan prosessoreita. Intel on julkaissut kirjaston [43], jolla kielimallien suoritus integroidulla näytönohjaimella onnistuu. Kirjasto on käytettävissä sekä LangChainin että Ollaman kanssa. Kirjaston koodirepositorion esimerkeissä näkyy, että 7B kokoisia malleja voidaan suorittaa hyvällä nopeudella integroiduilla näytönohjaimilla. Yrityksen arvioitavaksi siis jää, onko sovelluksen nopeus tyydyttävä heidän käyttötarkoituksiinsa ja onko uusien kannettavien hankkiminen ajankohtaista.

Toinen ratkaisu ongelmaan olisi kielimallien ajaminen yrityksen omistamalla palvelinlaitteella, jolla olisi riittävän hyvät komponentit generointipyyntöjen nopeaan suorittamiseen. Palvelimella tulisi olla vähintään yksi erillinen näytönohjain. Ollama tukee jo valmiiksi usean näytönohjaimen ja mallin käyttöä samaan aikaan sekä generointipyyntöjen jonoon laittamista [25]. Jono-ominaisuus sopii hyvin juuri yhteiskäyttöiselle palvelimelle, koska sille voisi tulla useita pyyntöjä samaan aikaan. Yrityksen tulee arvioida, paljonko laitteisiin panostetaan ja kumpi ratkaisu on parempi.

Sovelluksen käyttöliittymä on helppokäyttöinen, mutta myös hyvin pelkistetty. Käyttöliittymän ulkoasua voitaisiin vielä parannella ainakin kysymysten lähetyksen ja vastauksien esityksen osalta. Tekstin kappaleet voitaisiin erotella paremmin ja mahdolliset otsikot voitaisiin esittää isompana. Kysymys katoaa tällä hetkellä lähetyksen jälkeen. Se voitaisiin lisätä näkyviin jonnekin, kuten vastauksen yhteyteen. Toiminnallisuuksia voitaisiin myös lisätä helpottamaan kokemusta. Sovellukseen voitaisiin lisätä keskusteluhistoria, jotta kysymykset ja vastaukset pysyisivät löydettävissä. Samalla sovellus voisi käyttää näitä aikaisempia tekstejä uuden vastauksen generoinnissa.

Sovelluksen RAG-osuutta voitaisiin nopeuttaa muuttamalla logiikkaa siten, ettei joka vaihetta joutuisi tekemään uudestaan joka kysymyksellä. Esimerkiksi dokumentin käsittely voitaisiin tehdä vain kerran ja vektorisoidut osat voitaisiin pitää pysyvässä muistissa. Osien pilkkomisen logiikkaa voitaisiin myös muuttaa nykyisestä merkkeihin perustuvasta lähestymistavasta. Otsikoiden tai muiden semanttisesti järkevien kokonaisuuksien perusteella ositus voisi tuoda parempia tuloksia, mutta se saattaa vaatia dokumentin esikäsittelyä sopivaan muotoon. RAG-prosessiin voitaisiin lisätä Reranker-malli, joka arvioisi uudestaan samankaltaisten vektorien järjestyksen. Myös Embeddings-mallin sopivuutta voitaisiin kokeilla vaihtamalla se toisiin ja testaamalla tapahtuneita eroja koeajoissa. LangChainissa tällaisten osien vaihto on helppoa sen modulaarisuuden suunnittelun vuoksi. Myös tietolähteen formaatin vaihto toiseen on LangChainissa helppoa sen monien Loader-luokkien ansiosta.

Ollamalla on lisäasetuksia, joihin tässä projektissa ei koskettu. Esimerkiksi FlashAttention-tekniikan päälle kytkemisellä saataisiin laskettua muistinkäyttöä, jos laitteisto tukee sitä. Lisäasetuksissa voidaan myös säätää käytössä olevaa kontekstikokoa. Ollamalta löytyy myös suorituskykytestitila, jossa määritetylle kielimallille lasketaan erilaisia suoritusmetriikoita.

Lokaalien kielimallien tulevaisuusnäkymät ovat hyvät, kuten ovat kielimallien yleisestikin. Pelkästään suorituskykyloikka Llama 2 -malleista Llama 3 -malleihin oli suuri ja Meta luultavasti julkaisee Llama 4-mallit jälleen vuoden jälkeen edellisestä sukupolvesta. Kielimallien nopea kehitys povaa hyvää tulevaisuudelle, sillä mallit kehittyvät jatkuvasti paremmiksi ja nopeammiksi suorittaa. Yritysten kannattaa aloittaa kielimallien hyödyntäminen aikaisessa vaiheessa, koska kypsän tuotteen saamisessa kestää kauan. Osaamisen kasvaessa on helpompi siirtyä uudempiin kielimalleihin ja ratkaisuihin sitä mukaa, kun ne kehittyvät.

6 Yhteenveto

Insinöörityön tavoitteena oli tutkia lokaalien kielimallien hyödyntämistä yrityksen tarpeisiin tehdyssä sovelluksessa. Tilaajaksi määriteltiin hypoteettinen yritys, jonka motiivina käyttää lokaaleja kielimalleja oli yrityksen sensitiivisen sisäisen datan pitäminen yksityisenä. Yritys halusi tietää, voidaanko lokaaleilla kielimalleilla tehdä hyödyllinen sovellus, joka toimisi yrityksen käyttämillä työkannettavilla.

Insinöörityössä tutkittiin yleisesti tekoälyä ja kielimalleja yrityksen tavoitteiden näkökulmasta. Tutkimuksessa etsittiin tietoa kielimallien hyödyistä ja rajoitteista sekä syvennyttiin lokaalien kielimallien ja pilvimallien eroihin. Tutkimuksessa löydettiin Retrieval-Augmented Generation -menetelmä, jota hyödyntämällä yrityksen pitkistä PDF-dokumenteista saataisiin etsittyä relevanttia tietoa kielimallia varten.

Insinöörityön projektina luotiin MVP-versio (Minimum Viable Product) sovelluksesta eli yksinkertaisin mahdollinen sovellus, joka täyttää projektille asetetut vaatimukset. Sovelluksen toteutukseen valitut teknologiat osoittautuivat sopiviksi ja helppokäyttöisiksi sekä kehityksen aikana että jatkokehitystä ajatellen. Sovellus täytti asetetut vaatimukset suurimmilta osin hyvin, mutta ongelmaksi muodostui suoritusnopeus heikommalla laitteistolla. Yrityksen työkannettavien suorituskyky arvioitiin jonnekin käytettyjen laitteiden välille. Ratkaisuksi esitettiin uudemman laitteiston ostaminen sekä generoinnin siirtäminen erilliselle ja tehokkaalle palvelinlaitteelle. Yrityksen arvioitavaksi jäi, oliko suoritusnopeus riittävä vai tarvitaanko investointeja uuteen laitteistoon.

Viimeiseksi esiteltiin vaihtoehtoja jatkokehitykselle. Pelkistetylle sovelluksen käyttöliittymälle esitettiin parannuksia käyttäjäkokemukseen. Lisäksi esiteltiin toiminnallisuuksia, jotka helpottaisivat sovelluksen päätoimintoa eli kysymysten kysymistä. Lopuksi vielä arvioitiin lokaalien kielimallien tulevaisuutta ja kehotettiin aloittamaan kielimallien käyttäminen mahdollisimman pian.

Lähteet

1 Hu, Krystal. 2023. ChatGPT sets record for fastest-growing user base - analyst note. Verkkoaineisto. Reuters. <https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/>. Päivitetty 2.2.2023. Luettu 19.3.2025.

2 Stryker, Cole & Kavlakoglu, Eda. 2024. What is artificial intelligence (AI)? Verkkoaineisto. IBM. <https://www.ibm.com/think/topics/artificial-intelligence>. 9.8.2024. Luettu 19.2.2025.

3 What is a large language model? 2024. Verkkoaineisto. SAP. <https://www.sap.com/resources/what-is-large-language-model>. 2.7.2024. Luettu 25.2.2025.

4 What is artificial intelligence (AI)? Verkkoaineisto. Cloudflare. <https://www.cloudflare.com/learning/ai/what-is-artificial-intelligence/>. Luettu 25.2.2025.

5 Gottweis, Juraj & Natarajan, Vivek. 2025. Accelerating scientific breakthroughs with an AI co-scientist. Verkkoaineisto. Google. <https://research.google/blog/accelerating-scientific-breakthroughs-with-an-ai-co-scientist/>. 19.2.2025. Luettu 21.2.2025.

6 Khan, Arshad. 2024. Artificial Intelligence: A Guide for Everyone. Springer, Cham.

7 De Cremer, David; Morini Bianzino, Nicola; Falk, Ben. 2023. How Generative AI Could Disrupt Creative Work. Verkkoaineisto. Harvard Business Review. <https://hbr.org/2023/04/how-generative-ai-could-disrupt-creative-work>. 13.4.2024. Luettu 24.2.2025.

8 Georgieva, Kristalina. 2024. AI Will Transform the Global Economy. Let's Make Sure It Benefits Humanity. Verkkoaineisto. International Monetary Fund (IMF). <https://www.imf.org/en/Blogs/Articles/2024/01/14/ai-will-transform-the-global-economy-lets-make-sure-it-benefits-humanity>. 14.1.2024. Luettu 24.2.2025.

9 Hart, Jonny. 2025. AI won't take your job; it will make you better at it. Verkkoaineisto. Temple University. <https://news.temple.edu/news/2025-01-16/ai-won-t-take-your-job-it-will-make-you-better-it>. 16.1.2025. Luettu 25.2.2025.

10 Basiouny, Angie. 2024. AI Can't Replace You at Work. Here's Why. Verkkoaineisto. Knowledge at Wharton.

<https://knowledge.wharton.upenn.edu/article/ai-cant-replace-you-at-work-heres-why>. Päivitetty 3.7.2024. Luettu 25.2.2025.

11 Rosalsky, Greg. 2025. Why the AI world is suddenly obsessed with a 160-year-old economics paradox. Verkkoaineisto. NPR. <https://www.npr.org/sections/planet-money/2025/02/04/g-s1-46018/ai-deepseek-economics-jevons-paradox>. 4.2.2025. Luettu 26.2.2025.

12 What is generative AI? Verkkoaineisto. Cloudflare. <https://www.cloudflare.com/learning/ai/what-is-generative-ai>. Luettu 2.3.2025.

13 What is a large language model (LLM)? Verkkoaineisto. Cloudflare. <https://www.cloudflare.com/learning/ai/what-is-large-language-model/>. Luettu 25.2.2025.

14 Stryker, Cole. 2024. What is multimodal AI? Verkkoaineisto. IBM. <https://www.ibm.com/think/topics/multimodal-ai>. 15.7.2024. Luettu 3.3.2025.

15 Kulyk, Oleg. 2024. LLM Instruct vs Chat - A Comprehensive Analysis. Verkkoaineisto. ScrapingAnt. <https://scrapingant.com/blog/llm-instruct-vs-chat>. 27.6.2024. Luettu 10.3.2025.

16 LLM prompting guide. Verkkoaineisto. Huggingface. <https://huggingface.co/docs/transformers/en/tasks/prompting>. Luettu 6.3.2025.

17 OpenAI Guide: Prompt Engineering. Verkkoaineisto. OpenAI. <https://platform.openai.com/docs/guides/prompt-engineering>. Luettu 6.3.2025.

18 Jiang, Zhifeng; Jin, Zhihua; He, Guoliang. 2024. Safeguarding System Prompts for LLMs. Verkkoaineisto. <https://arxiv.org/abs/2412.13426>. Päivitetty 9.1.2025. Luettu 10.3.2025.

19 Sutinen, Martti; Rainio, Ida. 2024. Suomalaiset kielimallit kehittyvät vauhdilla - oletko valmis uudistamaan toimintaa? Verkkoaineisto. Futurice. <https://www.futurice.com/blog/suomalaiset-kielimallit-ja-uudistaminen>. 21.8.2024. Luettu 11.3.2025.

20 Cheng, Jeffrey; Marone, Marc; Weller, Orion; Lawrie, Dawn; Khashabi, Daniel; Van Durme, Benjamin. 2024. Dated Data: Tracing Knowledge Cutoffs in Large Language Models. Verkkoaineisto. <https://arxiv.org/abs/2403.12958>. Päivitetty 17.9.2024. Luettu 12.3.2025.

21 Bergmann, Dave. 2024. What is a context window? Verkkoaineisto. IBM. <https://www.ibm.com/think/topics/context-window>. Päivitetty 7.11.2024. Luettu 12.3.2025.

22 Helakallio, Anna. 2025. Pärjääkö tekoäly ohjelmoijille? Pelkkä nopeus ei riitä. Verkkoaineisto. Mikrobitti. <https://www.mikrobitti.fi/uutiset/mb/d1ba917f-a589-4c3f-8185-1c07ee5825cc>. 2.3.2025. Luettu 3.3.2025.

23 How-to guides: Prompting. Verkkoaineisto. Meta. <https://www.llama.com/docs/how-to-guides/prompting/>. Luettu 13.3.2025.

24 Clayton, Jesse. 2024. How to Accelerate Larger LLMs Locally on RTX With LM Studio. Verkkoaineisto. Nvidia. <https://blogs.nvidia.com/blog/ai-decoded-lm-studio/>. 23.10.2024. Luettu 16.3.2025.

25 Ollama Documentation. 2025. Verkkoaineisto. Ollama. <https://github.com/ollama/ollama/tree/main/docs>. Päivitetty 14.3.2025. Luettu 17.3.2025.

26 What is LangChain? 2023. Verkkoaineisto. IBM. <https://www.ibm.com/think/topics/langchain>. 31.10.2023. Luettu 19.3.2025.

27 Holdsworth, Jim; Kosinski, Matthew. 2024. What is a vector database? Verkkoaineisto. IBM. <https://www.ibm.com/think/topics/vector-database>. 29.7.2024. Luettu 19.3.2025.

28 Reinikainen, Samuli. 2024. Kurkistus hypetetyn tekoälytermin konepellin alle – Miten RAG toimii ja mitä sen hyödyntäminen vaatii? Verkkoaineisto. Efima. <https://www.efima.com/blogi/kurkistus-hypetetyn-tekoalytermin-konepellin-alle-miten-rag-toimii-ja-mita-sen-hyodyntaminen-vaatii>. 22.04.2024. Luettu 19.3.2025.

29 Maxwell, Timothy. 2023. Should You Use a Local LLM? 9 Pros and Cons. Verkkoaineisto. Make Use Of. <https://www.makeuseof.com/should-you-use-local-llms/>. 24.10.2023. Luettu 25.3.2025.

30 Bjarnason, Baldur. 2025. Poisoning for propaganda: rising authoritarianism makes LLMs more dangerous. Verkkoaineisto. <https://www.baldurbjarnason.com/2025/poisoning-for-propaganda/.> 3.2.2025. Luettu 25.3.2025.

31 Verspeurt, Joren. 2023. Staying compliant and in control of your data when using ChatGPT and other Large Language Models. Verkkoaineisto.

Superlinear. <https://superlinear.eu/insights/articles/staying-compliant-and-in-control-of-your-data-when-using-chatgpt-and-other-large-language-models>. 18.4.2023. Luettu 25.3.2025.

32 Wulff Wold, Jacob. 2025. Deafening Commission silence with no credible EU-US data oversight left. Verkkoaineisto. Euractiv. <https://www.euractiv.com/section/tech/news/deafening-commission-silence-with-no-credible-eu-us-data-oversight-left/>. 3.3.2025. Luettu 25.3.2025.

33 Ren, Shaolei; Tomlinson, Bill; Black, Rebecca W.; Torrance, Andrew W. 2024. Reconciling the contrasting narratives on the environmental impact of large language models. Verkkoaineisto. Nature. <https://www.nature.com/articles/s41598-024-76682-6>. 1.11.2024. Luettu 26.3.2025.

34 Minions: where local and cloud LLMs meet. Verkkoaineisto. Ollama. <https://ollama.com/blog/minions>. 25.2.2025. Luettu 26.3.2025.

35 Fourrier, Clementine; Habib, Nathan; Lozovskaya, Alina; Szafer, Konrad; Wolf, Thomas. 2024. Performances are plateauing, let's make the leaderboard steep again. Verkkoaineisto. Hugging Face. <https://huggingface.co/spaces/open-llm-leaderboard/blog>. 26.6.2024. Luettu 26.3.2025.

36 Use Ollama with any GGUF Model on Hugging Face Hub. Verkkoaineisto. Hugging Face. <https://huggingface.co/docs/hub/ollama>. Päivitetty 19.12.2024. Luettu 26.3.2025.

37 Tantai, Hengtao. 2024. From Smooth Chatting to Precise Execution: differences between “chat” and “instruct” modes in large language models. Verkkoaineisto. Medium. <https://medium.com/@zergtant/from-smooth-chatting-to-precise-execution-differences-between-chat-and-instruct-modes-in-9c6f73fc175e>. 16.5.2024. Luettu 27.3.2025.

38 Zem, Gregory. 2023. Understanding Permissive Licenses for Large Language Models (LLMs). Verkkoaineisto. Medium. <https://medium.com/@mne/understanding-permissive-licenses-for-large-language-models-llms-843d40909ce0>. 11.7.2023. Luettu 27.3.2025.

39 Introducing Llama 3.1: Our most capable models to date. Verkkoaineisto. Meta. <https://ai.meta.com/blog/meta-llama-3-1/>. 23.7.2024. Luettu 27.3.2025.

40 Nalpas, Maud. 2024. Understand LLM sizes. Verkkoaineisto. Google. <https://web.dev/articles/llm-sizes>. Päivitetty 30.5.2024. Luettu 27.3.2025.

41 Laine, Kari; Kostin Mansi, Helena. 2024. Tekoälyllä potentiaalia tehostaa lausuntokierrosten analysointia. Verkkoaineisto. Valtioneuvosto. <https://valtioneuvosto.fi/-/tekoalylla-potentiaalia-tehostaa-lausuntokierrosten-analysointia>. 29.8.2024. Luettu 27.3.2025.

42 FastAPI-dokumentaatio. Verkkoaineisto. <https://fastapi.tiangolo.com/>.

43 Ipex-llm, Intelin koodirepositorio. Verkkoaineisto. Github. <https://github.com/intel/ipex-llm>. Luettu 2.4.2025.